When the Team Writes Itself: Reliable and Traceable LLM-Assembled Agent Teams through Checked Model Transformations

Abstract

Team builders let a large language model (LLM) design a team of LLM agents for each task. Such teams often report success when they have failed, and their failures are hard to explain. The cause is a *composition gap*: agents are defined only by prose roles, hand-offs between them are free text, and nothing checks that the parts fit together before the team runs. We present AutoM2M, which brings model-driven engineering to team building. A builder LLM proposes a *typed team*: each agent has a metamodel as its interface, and each hand-off is a model-to-model transformation. Our runtime, AgentHOT, executes these hand-offs, with LLMs filling in only individual, validated values. A checker admits a team only if six well-formedness rules hold. Trace models make failures easy to locate, and every repair must pass the same checker. We evaluated AutoM2M in 44,352 runs on ClassEval and HumanEval+ with seven open-weight models under eight conditions, plus fault injection, independently seeded defects and mutation analysis. AutoM2M had the fewest silent failures (about 6%, against 18–20% for a single agent and 27–56% for other multi-agent teams), partly through abstention; admitted teams alone still fail silently about half as often as a single agent. Its traces located 96–98% of injected faults, and its checker caught 63% of seeded defects in milliseconds, which LLM critics could not do. The cost is 7–16 times the output tokens of a single agent and lower task success. Checked interfaces do not make LLM-built teams solve more tasks, but they make their claims of success more trustworthy and their failures traceable.

Keywords: LLM-based multi-agent systems, team builders, model-driven engineering, model-to-model transformation, ATL, static analysis, failure attribution, empirical study

1 Introduction

Software is increasingly produced by *teams* of large language model (LLM) agents. Each agent is an LLM that runs in a loop with tools and a role, and the team divides analysis, design, implementation and testing among its members [**? ?**]. The first such teams were designed by hand [**? ? ?**]. A second generation removes the human designer. *Per-task team builders* read a task, decide which agents to create, write their role

prompts and wire who talks to whom [? ? ? ?]; offline methods search over agentic workflows [? ? ?]; and orchestrators trained by reinforcement learning pick the next agent, write workflows or emit whole teams [? ? ?].

These teams fail often, and their failures are hard to explain. Cemri et al. annotated more than 1,600 traces of seven multi-agent frameworks and found that system-design issues (44.2% of failure-mode occurrences) and inter-agent misalignment (32.3%) outnumber task-verification problems (23.5%) [?]. Lin et al. locate errors on the *edges* of a team, the hand-offs, where information is dropped, corrupted or re-interpreted [?]. Even naming the culprit is hard. On the Who&When benchmark of annotated multi-agent failures, the best automated methods identify the responsible agent in 53.5% of cases and the decisive step in 14.2% [?]. With full input traces, agent accuracy rises only to about two thirds and step accuracy to about one third [?].

The composition gap. We hypothesise that part of these failures has a cause familiar to every software engineer: the parts of the team have no interfaces. A team builder composes a system out of components (agents) and connectors (hand-offs). Yet each component is specified only by a natural-language job description, every hand-off is free text that the next LLM re-interprets, and nothing checks that the parts fit before the team runs. A deterministic audit of the 126 automatically assembled teams of Who&When (Section **??**) illustrates the point: none of the 356 generated role prompts names a teammate or states what its agent must produce, and no team plan assigns a step to a named role. These specifications are not wrong by the standards of their builder, which never promised such interfaces; they simply leave nothing that a program could check. Software architecture calls this situation *architectural mismatch*: components that work alone fail together because of unchecked assumptions about each other [? ?]. Its classical remedies are typed interfaces, contracts and specified connectors [? ?]. We call the absence of such interfaces in LLM agent teams the *composition gap*. It gives rise to *composition defects*, such as parts of the task that no agent owns, hand-offs whose content the receiver cannot use, artefacts with two writers, completion that is merely claimed, and work routed to agents without the needed tools. In current team builders, such defects are neither prevented before execution, nor detected during it, nor attributable after it.

Key insight. Model-driven engineering (MDE) already offers checkable interfaces for artefacts that are derived from one another. A metamodel states what an artefact may contain, a model-to-model (M2M) transformation states rule by rule how one artefact is derived from another [? ?], and static analysis can reject an ill-typed or incomplete transformation before it runs [?]. We therefore treat each agent's output as a model, each hand-off as a transformation, and the team as a network of transformations. Because an LLM, not a person, now writes these metamodels and transformations, the composition gap moves up one level: from the messages to the *team specification*, which must itself be checked.

This paper. We present AutoM2M (*automatic model-to-model teams*), which applies one principle at two levels: *LLMs propose; programs decide.*

At the level of hand-offs, AutoM2M's execution layer AgentHOT executes *hybrid* transformations: a deterministic engine fixes which objects exist and how they are

linked, and an LLM fills in only the values that no formula can compute, each from a declared input and behind a validator.

At the level of the team, an LLM proposes a *typed team*, and a deterministic checker decides whether the team may run. The checker evaluates six well-formedness rules, the *admission rules* W1–W6, on the specification alone; each rules out a composition defect or a part of one. We call the builder's proposals and revisions, the checker's verdicts and the checked repairs together the *build–check–repair loop*.

When an admitted team fails, its *trace models*, which record for every value the rule, agent and inputs that produced it, locate the failed value; a bounded replay of that value classifies the fault; and every repair must pass the same checker before it takes effect. We use the rule language of ATL as notation and as semantics; our implementation is a Python engine that interprets these rules rather than an ATL/EMF tool chain (Section **??**). We keep two names because the parts are separable: AgentHOT executes any typed team, whether written by an LLM or by a person, and AutoM2M is the approach that builds, checks and repairs teams on top of it.

What an engineer should get. Our goal is not primarily higher task success. Two properties matter at least as much when a team's output is used without a human re-checking every result. The first is *reliability*: when the team says it is done, the result should be correct, and when it fails, it should say so. A failure that is reported as a success, a *silent failure*, is the costly case, because nobody looks at it. The second is *traceability*: when the team fails, an engineer should be able to find the responsible part, understand what went wrong and repair that part without starting over. Both properties are standard expectations of conventional software, and both are missing from free-form agent teams, whose "done" is a message that an agent chooses to send and whose only record is a chat transcript.

AutoM2M is designed to provide both. Its completion signal is computed by the engine from executed checks that the checker has shown to cover the goal, and its traces link every value to the rule, agent and inputs that produced it. Both rest on the checker, which must catch composition defects before the team runs. This machinery has a price in tokens, time and, for builders that cannot write an admissible team, lost work. We therefore ask three research questions, each about one benefit and its price:

- **RQ1** (Reliability): When an AutoM2M team reports that it is done, can we believe it, and when it fails, does it say so?
- **RQ2** (Traceability): When an AutoM2M team fails, can the failure be traced to the responsible part of the team, explained and repaired there?
- **RQ3** (Prevention): Does the admission check catch composition defects before the team runs, and what does it cost?

We answer them with 44,352 runs of eight conditions on ClassEval and HumanEval+ with seven open-weight models (RQ1), 1,108 injected faults compared with four transcript-based attribution methods (RQ2), and 275 independently seeded defects compared with LLM critics, plus 151,516 mutants of LLM-written teams (RQ3). We then report the price in task success, including a pre-registered experiment that tested whether a larger budget closes the gap.

Contributions.

- **C1** *A problem analysis* that grounds five composition defects (D1–D5; the six admission rules cover them, with D2 split over two rules) in published failure taxonomies and in an audit of 126 automatically built teams (Section **??**).
- **C2** *A formulation* of LLM agent teams as networks of hybrid M2M transformations, built on standard MDE notions: the semantics of hybrid hand-offs, the AgentHOT runtime that executes them, and typed teams as their checkable specifications (Section **??**).
- **C3** *Six admission rules.* Three of them (W1–W3) adapt established static checks of model transformations to LLM-written teams; the other three are new for agent teams, most notably a two-sided, feature-level *anchoring* criterion for coverage (W4) and engine-decided completion (W5). We add two propositions on their cost and on the structural coverage they guarantee, and an attribution procedure that turns failure localisation into a lookup plus a bounded replay (Section **??**).
- **C4** *An empirical evaluation* of reliability, traceability and prevention against single-agent, compute-matched single-agent, free-form, critic and shared-schema baselines and two ablations, with fault injection, independently seeded defects, mutation analysis and a pre-registered budget experiment (Section **??**).
- **C5** *A replication package* with the checker, the runtime, the team fixtures, the analysis pipeline and all raw data (Section **??**).

Summary of findings.

- *Reliability.* AutoM2M has the fewest silent failures of the eight conditions on both benchmarks (5.9% and 5.6% of runs). Part of this is abstention, but admitted teams alone still fail silently about half as often as a single agent. A compute-matched single agent that runs the task's docstring examples before stopping fails silently more often than a plain one. The price of AutoM2M's caution is that more than half of its correct ClassEval results are not reported as done.
- *Traceability.* The trace locates the responsible binding of 96–98% of injected faults and names the fault class about twice as often as the best transcript-based method. Repairing the diagnosed part is as successful as rebuilding the team, with 45% fewer tokens.
- *Prevention.* The checker is the only judge that separates independently seeded defective teams from clean ones; only its two-sided anchoring rejects teams whose producers never see the goal.
- *Price.* AutoM2M uses 7–16 times the output tokens of a single agent and succeeds less often (10.6% against 24.5% on ClassEval), because weak builders often write no admissible team and typed teams lose success on class-level tasks. A post-hoc revision recovers much of this loss on held-out tasks but, with a weak model, brings back silent failures; a larger sampling budget does not close the gap (pre-registered).

Structure. Section **??** introduces LLM agent teams, the running example, the composition defects and the MDE notions we build on. Section **??** presents AutoM2M.

Section ?? discusses related work, Section ?? reports the evaluation, including the implementation and how it realises each algorithm (Section ??), Sections ?? and ?? discuss implications and threats to validity, and Section ?? concludes.

2 Background and Running Example

AutoM2M sits between LLM-based multi-agent systems and model-driven engineering. We therefore start on the agent side: what an agent team and a team builder are (Section ??), a running example of a team that a builder produces (Section ??), and the composition defects such a team can contain (Section ??). We then introduce the MDE notions on which our remedy rests: models, metamodels and conformance (Section ??), and model-to-model transformations in ATL (Section ??).

2.1 LLM agents, teams and team builders

An LLM's output is sampled, so the same prompt can yield different answers, some fluent yet wrong [? ?]. An *agent* runs an LLM in a loop with tools until the LLM declares that it is done [? ?]. To say precisely what a team builder outputs, we need a minimal notion of a team.

Definition 1 (Agent team, hand-off) An agent team is a tuple $(A, \rho, \kappa, E, \mathit{stop})$ of a set of agents A, a role description $\rho(a)$ and a tool set $\kappa(a)$ for each agent $a \in A$, a set of communication edges $E \subseteq A \times A$, and a termination rule *stop*. A *hand-off* is the passing of one agent's output to another agent along an edge in E.

In most frameworks, ρ is free text, every hand-off is a natural-language message, and *stop* is either an agent's declaration (for example the token `TERMINATE`) or a turn limit [? ?]. A *team builder* is a procedure, typically an LLM or a learned policy, that maps a task to an agent team. Builders are optimised by end-to-end scores or by an LLM's judgement of a transcript; Section ?? reviews them.

2.2 Running example

We use one small task throughout the paper. It has the shape of a ClassEval task [?], the class-level one of our two benchmarks (Listing ??).

The task has three methods and three docstring examples. We write E$i.j$ for the j-th example of method i. Two details matter later. First, the examples of `mark_done` (E2.1 and E2.2) run on the board that E1.1 created, so they depend on its state. Second, *E2.2* is the example that requires a `ValueError` on a repeated call, and `export_csv` has no example at all. ClassEval's hidden tests check every method, including the second call to `mark_done`.

Listing ?? shows a team that a free-form builder in the style of CaptainAgent produces for this task. The role prompts are well written, and the plan is reasonable. Section ?? shows that every composition defect we study is nevertheless latent in it.

In Section ??, the same task is solved by a *typed* team that we call DevTeam. It has an Architect who states a contract for each method, a Tester who writes unit

Listing 1 Raw input: the `TaskBoard` task as given to every condition (abridged).

```
class TaskBoard:
    """A small in-memory to-do board."""
    def add_task(self, title):
        """Add a task and return its integer id. Raises ValueError if title is empty.
        >>> b = TaskBoard(); b.add_task('write report')
        1"""
    def mark_done(self, task_id):
        """Mark a task as done. Raises ValueError if the task is already done.
        >>> b.mark_done(1)
        >>> b.mark_done(1)
        Traceback (most recent call last): ... ValueError"""
    def export_csv(self):
        """Return all tasks as CSV text with header 'id,title,done'."""
```

Listing 2 Raw input: a free-form team for `TaskBoard` (role prompts abridged; generated in the style of CaptainAgent [**?**]).

```
Python_Expert:       "Expert in Python classes and clean, idiomatic implementations ..."
QA_Expert:           "Expert in designing thorough test suites and edge cases ..."
Verification_Expert: "Vigilantly verify that the final solution is correct ..."
                     tools: [search]
## Plan  1. Understand the class.  2. Implement the methods.  3. Test.
##       4. Verify and reply TERMINATE.
## Output format: the complete class.
```

tests from each method's examples, and a Developer who implements each method. Its views, rules, a defective first proposal and the checker's diagnostics are introduced step by step (Listings **??**–**??**).

2.3 Composition defects

Published failure analyses describe *what* goes wrong in multi-agent runs. To design checks, we need to know *where in the team's specification* a failure originates. We therefore read the published evidence as five defects that a team builder can introduce when it composes agents, plus one consequence.

Definition 2 (Composition defect) A composition defect is a property of an agent team's *specification*: which agent owns which work, what is handed over, which tools are where, and how completion is decided. It is not a property of what one agent produces from a given input. A specification has a composition defect if even agents that answer every request correctly cannot make the team deliver its task.

Table **??** lists the five defects (D1–D5) and the consequence C, together with the published failure modes that each subsumes. The mapping from failure modes to defects is analytical; we do not estimate how often each defect decides a failure (Section **??** explains why), but test whether the checker catches each defect (RQ3). Single-agent reasoning errors (MAST FM-1.1, 2.2, 2.6) are not composition defects, because no composition can make an individual LLM reason correctly.

The audit column of Table **??** was produced by a deterministic script. For each of the 126 failures of automatically assembled teams in Who&When, it reads the

Table 1 Composition defects of automatically assembled teams. FM-$x.y$: MAST failure modes [**?**] (names listed there); DG/SC/RD/CG: data gap, signal corruption, referential drift and capability gap in the edge-level taxonomy of AgentAsk [**?**]. The audit column refers to the 126 CaptainAgent teams of Who&When [**?**].

Defect	What goes wrong	Published modes	Audit evidence
D1 Coverage gap	Part of the task is owned by no agent, or no check ever uses the goal's data.	DG; FM-2.4, FM-3.2	0/126 plans assign any step to a named role
D2 Hand-off mismatch	What a producer emits is not what its consumer needs, in form, reference or content.	SC, RD; FM-1.4, 2.1, 2.5	0/356 roles state their output
D3 Ownership conflict	Two agents act on the same artefact, or none knows it is theirs.	FM-1.2, FM-1.3	0/356 roles name a teammate
D4 Unverifiable completion	"Done" is a claim in the conversation, not a checked property.	FM-1.5, 3.1, 3.3	82/126 failed runs ended with a `TERMINATE`
D5 Capability mismatch	Work is routed to an agent that lacks the needed tool or skill.	CG	search-only verifiers check code
Consequence			
C Opaque attribution	A failure cannot be traced to the defect or agent that caused it.	–	53.5%/14.2% agent/step accuracy [**?**]

generated role prompts and the manager brief (the plan that CaptainAgent writes) and records only what the specification *states*. The audit therefore shows what the specifications lack, not that the lack caused each failure. We do not claim such a causal link anywhere in this paper (Section **??** explains why).

Defects in the running example. The free-form team of Listing **??** contains every defect of Table **??** in latent form. No role says what it produces (D2) or names a teammate (D3). Nothing requires that E2.2 ever reaches a test (D1). The Verification_Expert must check code but has only a search tool (D5). And the run ends when the Verification_Expert replies `TERMINATE` (D4).

2.4 Models, metamodels and conformance

Our remedy for the composition gap is to give every agent a precise interface. MDE provides one: a metamodel, which states what a model may contain, and conformance, which checks mechanically that a model stays within it. We use metamodels as the "forms" that agents fill in, and conformance as the first and cheapest check that what an agent produced is admissible.

MDE treats models, not code, as the primary artefacts of development and relies on programs that check and transform them [**? ? ?**]. A model *represents* an original for a purpose and keeps only the properties that serve it; a metamodel is a model of

the *modelling language* in which such models are written [**?**]. A model *conforms to* its metamodel [**?**]; this is a *linguistic* instance-of relation, between a model element and the language concept it instantiates [**?** **?**]. Definitions **??** and **??** follow the usual reading of metamodels as type graphs with constraints and of models as typed attributed graphs [**?**]. Because each agent will own its own metamodel, but all of them must refer back to the task, we also allow references from one metamodel into another.

Definition 3 (Metamodel) A metamodel is a tuple $MM = (C, Att, Ref, \preceq, mult, \Phi)$ where C is a finite set of classes; Att a set of attributes, each owned by a class $c \in C$ and typed by a primitive type; Ref a set of references $r : c \to c'$ between classes, some of which are *containments* (the target object is owned by the source object); $\preceq$ a generalisation order on C; $mult$ a function that assigns each attribute and reference a multiplicity $[l..u]$; and Φ a set of well-formedness constraints in the Object Constraint Language (OCL) [**?**]. A reference may also be *external*, $r : c \to MM'.c'$, typed by a class of another metamodel MM'. A *feature* of c is an attribute or reference whose source is c or a superclass of c.

Definition 4 (Model, conformance) A model M over MM is a finite object graph: a set of objects, each typed by a class of C, with values (slots) for its features. M *conforms to* MM, written $M \ \chi \ MM$, iff every slot respects its feature's type and multiplicity, every object has at most one container and containment has no cycles, every constraint in Φ holds, and every external reference points to an object of the right class in the referenced model. $\mathcal{M}(MM) = \{M \mid M \ \chi \ MM\}$ is the set of models of MM.

In the running example, the task of Listing **??** is turned into a model with one Task, three Method and three Example objects. Its metamodel, the *goal view*, appears in Listing **??** (Section **??**), together with the views of the three agents. The reference `MethodDesign.method : Goal.Method` in the Architect's view is external: it points from a design object to the goal method it designs.

Conformance fixes what *can* be stated, not whether the statement is right. We call an agent team "typed" in this limited sense: its interfaces are metamodels, so whether the parts fit together becomes a question about the features that each part declares, reads and writes (made precise in Section **??**), rather than about prose. We do not use model subtyping [**?**]; every view is used exactly as declared.

2.5 Model-to-model transformations and ATL

A hand-off between agents derives one artefact from others, for example a design from requirements or code from a design. MDE describes such derivations as model transformations, "the heart and soul" of MDE [**?**]. A model transformation automatically produces a model from one or more source models [**?** **?**]. It is *exogenous* if source and target metamodels differ [**?**], and its *intent* fixes the properties it must satisfy, such as completeness [**?**]. In AutoM2M, a hand-off may read the outputs of several agents but writes into one agent's view, which motivates the following restricted form.

Definition 5 (M2M transformation) An M2M transformation is a relation $T \subseteq \mathcal{M}(MM_{s_1}) \times \cdots \times \mathcal{M}(MM_{s_k}) \times \mathcal{M}(MM_t)$ between models of $k \geq 1$ source metamodels and models of one

target metamodel, specified by a set of rules. It is *functional* if every tuple of sources is related to exactly one target.

A hand-off in a software team is therefore an exogenous, unidirectional transformation whose intent includes completeness: everything the sources state must be reflected somewhere in the target. In the running example, the Developer's hand-off `Design2Impl` reads the Architect's and the Tester's views and writes the Developer's view.

We build on the rule language and execution model of ATL [**?** **?**], a transformation language that combines declarative and imperative constructs, restricted to its declarative core. ATL transformations are functional. The basic unit of an ATL transformation is the matched rule.

Definition 6 (Matched rule) A matched rule is a tuple $r = (p_r, g_r, t_r, B_r)$ where the *source pattern* p_r is a list of typed variables $x_i : C_i$ over source classes, the *guard* g_r is a Boolean OCL expression over them, the *target pattern* t_r is a list of typed variables over target classes, and the *bindings* B_r are assignments $f \leftarrow e$ of an OCL expression e to a feature f of a target variable. A *match* of r is an assignment of source objects to p_r that satisfies g_r.

In the running example, the Tester's rule `Method2Test` (Listing **??**) has the source pattern `m : Goal!Method`, no guard, the target pattern `t : Test!TestCase`, and among its bindings `name <- m.name`. Each goal method is a match, so the rule creates one test case per method.

ATL executes matched rules in two phases [**?**]. The *matching* phase computes every match, allocates its target objects and records a trace link. The *initialisation* phase evaluates the bindings. When a reference binding yields a source object, ATL *resolves* it: the target receives the object that was created from that source object, not the source object itself. Sources are read-only and targets are write-only. ATL also requires, and checks at run time, that each tuple of source objects is matched by at most one rule, so the result does not depend on rule order. Under this assumption, the target's *structure* is a deterministic function of the sources, and every target object is accounted for by a trace link. Many errors, such as ill-typed navigation, unresolved bindings, rule conflicts and missing mandatory features, can then be found statically [**?**].

Definition 7 (Trace link and trace model) Let $\mathcal{R}_T$ be the rules of a transformation T and $Match_T$ the set of their matches in one execution. A trace link is a triple (m, t, r) of a match m, the set t of target objects created for it, and the rule $r \in \mathcal{R}_T$. The *trace model* $TL_T \subseteq Match_T \times 2^{Obj_t} \times \mathcal{R}_T$, where Obj_t is the set of objects of the target model, is the set of trace links of that execution, persisted as a model in its own right [**?** **?**].

ATL's trace links can be exported as models [**?**] or generated as first-class, queryable models [**?**], and incremental execution re-runs only the rules whose matches changed [**?**]. AutoM2M relies on both: traces locate failures, and incrementality lets the runtime recompute only values whose inputs changed.

Two further notions name what AutoM2M manipulates as a whole. A *megamodel* is a model whose elements are models, metamodels and transformations [**?**]; a typed team is one. A *higher-order transformation* takes or produces a transformation model [**?**]; AutoM2M's compiler, which turns a typed team into executable rule modules, is one in this sense. The name AgentHOT abbreviates *Agent Hand-Off Transformations*.

3 Approach

AutoM2M applies one idea at two levels. At the level of hand-offs, its execution layer AgentHOT lets an LLM write individual *values* while a deterministic engine owns the *structure* of each hand-off. At the level of the team, AutoM2M lets an LLM *propose* a team while a deterministic checker owns the decision to *admit* it. Section **??** gives an overview. The following sections introduce the parts in the order in which each builds on the previous one: hybrid hand-offs, the language in which teams are written (Section **??**); typed teams, which a builder proposes (Section **??**); how an admitted team runs (Section **??**); the admission check (Section **??**); how a builder uses that check (Section **??**); and how failures are attributed and repaired (Section **??**). Implementation details are reported with the evaluation (Section **??**).

3.1 Overview: components and flow

Figure **??** shows the eight components of AutoM2M. The compiler and the runtime (⑤, ⑥) form AgentHOT, which executes a typed team. The other components build, check, diagnose and repair the team. As in any model-driven tool chain [**? ?**], the artefacts that flow between components are *models* that conform to metamodels (the goal model, the typed team, trace models and fault reports) and *transformations* (hand-offs and the rule modules compiled from them). The compiler is a higher-order transformation [**?**], since it produces transformations, and so is a repair delta whenever it changes hand-offs.

① **Lifter.** Parses the task text into a *goal model*, which records the task's classes, methods and examples. This is a deterministic text-to-model *injection*, that is, parsing text into a model [**? ?**].

② **Builder.** The only component that designs. Its *Proposer* samples several candidate teams, each constrained to the typed-team format (step A), and the checker ranks them (step B). Its *Reviser* then rewrites only those parts of the best candidate that the checker's diagnostics name (step C; Section **??**).

③ **Typed team.** The builder's output is a model, not a set of role prompts. It states the agents and their views, the hand-offs between views, who may write what, which tools each agent has, what must be covered and what is delivered, and when the run is done (Section **??**). Because it is a model, a program can analyse it before it runs [**?**].

④ **Checker.** Decides whether a team is admitted by six admission rules, W1–W6, which cover typing, single writers, completeness, coverage, completion and tools (step B; Section **??**). Its diagnostics name the offending element, and their number ranks the builder's candidates. A team that is still rejected after k_{adm} revision rounds is refused (NotAdmitted).

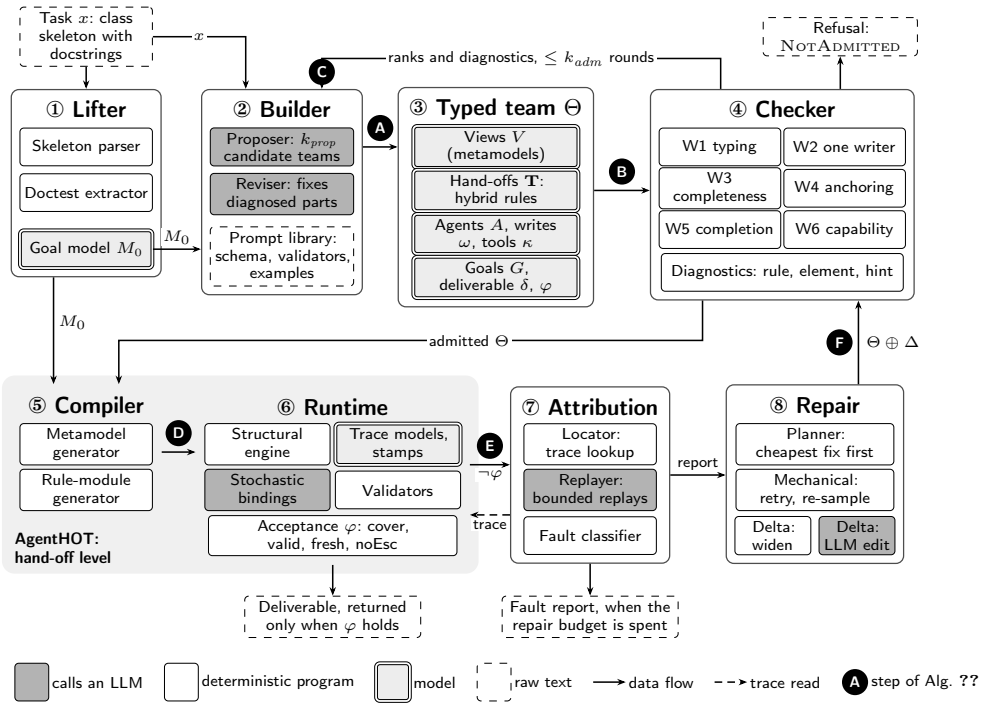


Figure 1 Components of AutoM2M (①–⑧) and the steps A–F of Algorithm **??**. The top row works on the *team*; the shaded row is AgentHOT, which executes *hand-offs*. A the builder proposes candidate teams; B the checker evaluates them; C its diagnostics drive at most k_{adm} revision rounds; D the admitted team is compiled and run; E if the acceptance predicate φ fails, attribution reads the trace models; F a repair delta Δ is applied only if the checker admits the edited team $\Theta \oplus \Delta$. φ is evaluated by the engine from four clauses: every goal object is covered by a checked value (cover), every accepted value is still valid (valid), no value is outdated (fresh), and no value failed for good (noEsc). In AutoM2M, no team reaches the compiler without passing the checker; the ablation Typed-NC is the exception, and the post-hoc extension R1 of Section **??** is applied after the check.

⑤ **Compiler.** Generates executable metamodels and rule modules from an admitted team and loads the goal model (step D). In MDE terms it is a *synthesis* higher-order transformation.

⑥ **Runtime.** Executes the hand-offs (Section **??**). The engine creates all objects and links. For each value that no formula can compute, it asks the LLM of the agent that owns the target view, and a validator accepts or rejects the value. Agents never converse with each other. The engine, not an agent, decides when the run is done.

⑦ **Attribution.** When the run is not done (step E), the locator follows trace links to the value that failed, and a bounded replay of that value decides why it failed (Section **??**).

⑧ **Repair.** Chooses the cheapest repair for each fault report. A *mechanical* repair re-samples a value (a retry) and leaves the team unchanged. Every other repair is a *delta*, an edit of the team model [**? ?**]: either an automatic one, which widens a

Table 2 MDE concepts on which AutoM2M is built, their counterpart in an agent team, and what changes when an LLM is the author. Circled numbers refer to the components of Figure **??**.

MDE concept (sources)	Counterpart in AutoM2M	What changes with an LLM author
Metamodel, conformance [**? ? ?**]	An agent's *view* ③; the goal view MM_0	views are proposed by an LLM, so they must be checked (W1–W3)
Exogenous M2M transformation, rules [**? ? ?**]	A *hand-off* ③, executed by the runtime ⑥	values are sampled, not computed
Relational semantics [**? ? ?**]; incremental propagation [**?**]	Meaning of a hand-off as a set of admissible targets (Def. **??**); re-running stale bindings	one direction only; membership is decided by validators
Transformation chain, correspondence [**? ?**]	The *team* as an acyclic network of hand-offs ③	coverage of goals along the network must be shown (W4)
Traceability [**? ? ?**]	Persisted trace models with stamps ⑥	traces also locate failures ⑦
Static analysis of transformations [**?**]	Admission W1–W6 ④	applied to every LLM proposal, inside the loop
Megamodel, higher-order transformation [**? ?**]	The typed team, whose elements are views and hand-offs; its compilation ⑤	the megamodel itself is LLM-authored
Edit operations, model versioning, co-evolution [**? ? ? ?**]	Repair *deltas* on the team model ⑧, admitted only if $\Theta \oplus \Delta$ is admissible ④	proposed by an LLM; preservation is re-checked, not assumed

footprint by the paths that attribution found, or one proposed by the builder's LLM. A delta takes effect only after the checker admits the edited team (step F).

AutoM2M invents no new modelling concepts. It reinterprets an agent team in established MDE terms and asks which classical guarantees survive when LLMs write the transformations and produce their values. Table **??** summarizes this correspondence. Two findings from the MDE literature shaped the design. First, developers and experts rank *traceability* among the transformation-language capabilities that matter most [**? ?**]; AutoM2M keeps traces as first-class models and uses them not only for change propagation but also for attribution. Second, model evolution is described by edit operations whose effect can be expressed at the level of the modeller's intent and traced across metamodel and model layers [**? ? ?**]; AutoM2M's repair deltas are such operations on the team model.

Algorithm **??** summarizes how the components cooperate. It is the entry point of AutoM2M; the procedures it calls are detailed in the following sections, and the letters in its comments are the steps of Figure **??**.

How to read Algorithm ??. Lines 1–2 turn the task into a goal model and ask the builder for a team; Build returns the best team it found together with the checker's diagnostics on it. If diagnostics remain (lines 3–4), AutoM2M refuses: the session ends without running anything, and therefore without claiming success. Otherwise, the loop of lines 5–13 runs the team at most k_{rep}+1 times. RunTeam (line 6) compiles the team, executes its hand-offs to a fixpoint and evaluates the acceptance predicate

Algorithm 1 AutoM2M: build, admit, run, attribute and repair a team for task x

Require: task x; building budgets $k_{prop}, k_{rev}, k_{adm}$; repair rounds k_{rep}
Ensure: the deliverable, or a refusal, or fault reports

1 $M_0 \leftarrow \textsc{Lift}(x)$ ▷ *goal model; deterministic*
2 $(\Theta, diag) \leftarrow \textsc{Build}(x, M_0)$ ▷ *steps A–C; Alg.* **??**
3 **if** $diag \neq \emptyset$ **then**
4 **return** $\textsc{NotAdmitted}(diag)$ ▷ *no team runs, nothing is claimed*
5 **for** $j = 0, 1, \ldots, k_{rep}$ **do**
6 $(done, F) \leftarrow \textsc{RunTeam}(\Theta, M_0)$ ▷ *step D; Alg.* **??**
7 **if** *done* **then return** the deliverable assembled from Θ's values
8 **if** $j = k_{rep}$ **then break**
9 $R \leftarrow \{\textsc{Attribute}(f) \mid f \in F\}$ ▷ *step E; Alg.* **??**
10 $(retry, \Delta) \leftarrow \textsc{PlanRepair}(R)$ ▷ *mechanical repairs and a delta*
11 **if** $\Delta \neq \emptyset$ **and** $\textsc{Admit}(\Theta \oplus \Delta) = \emptyset$ **then** ▷ *step F; Alg.* **??**
12 $\Theta \leftarrow \textsc{Apply}(\Theta, \Delta)$ ▷ *in place, by extension or rebuild*
13 mark the values in *retry* for re-sampling
14 **return** $\textsc{Failed}(R)$ ▷ *fault reports of the last round*

φ; it returns whether φ holds and the set F of failures, each of which names a failed clause and, where possible, a value of the team. If φ holds, the run is done (line 7). Otherwise, every failure is attributed (line 9) and a planner turns the fault reports into two kinds of repair (line 10): *mechanical* ones, which re-sample a value without changing the team (a retry or the re-sampling of an upstream value), and a *delta* Δ, which edits the team model (an automatic widening of a footprint, or an edit proposed by the builder's LLM). Only the delta changes the specification, so only the delta passes the checker (line 11); a delta that the checker rejects is discarded and Θ stays as it was. $\Theta \oplus \Delta$ denotes the edited team; Apply (line 12) installs it in the running team in one of three modes (Section **??**). Line 13 marks the values to be re-sampled in the next run, and if the last round still fails, the fault reports are returned (line 14). In our implementation, line 9 attributes at most two failures per round (Section **??**).

The running example through Figure ??. The lifter turns `TaskBoard` into a goal model with one task, three methods and three examples. The builder's first proposal for DevTeam has an Architect, a Tester and a Developer, but contains one instance of each defect; the checker rejects it with five diagnostics, and the reviser repairs the named parts until the team is admitted (Section **??**). The compiled team then runs. The implementation of `mark_done` fails, attribution traces the failure to the Tester's input, and an automatic delta widens that input, after which the engine declares the run done (Section **??**).

3.2 Hybrid hand-offs

This section defines the language in which every hand-off of a typed team is written: views as the agents' interfaces, and hybrid rules that split each hand-off between a deterministic engine and an LLM.

Listing 3 Goal view and agent views of the typed DevTeam in KM3 notation [**?**]; `Goal.Method` is an external (cross-view) reference.

```
package Goal {                                                  -- MM0, filled by Lift
  class Task       { attribute name : String;  attribute description : String;
                     attribute outline : String;                -- skeleton without doctests
                     reference methods [1-*] container : Method;
                     reference examples [0-*] : Example; }      -- in docstring order
  class Method     { attribute name : String;  attribute signature : String;
                     attribute docstring : String;  reference task : Task;
                     reference examples [0-*] container : Example; }
  class Example    { attribute call : String;  attribute expected : String; } }
package Design { class MethodDesign { attribute name : String;  attribute contract : String;
                                      reference method : Goal.Method; } }
package Test   { class TestCase     { attribute name : String;  attribute code : String;
                                      reference method : Goal.Method; } }
package Code   { class MethodImpl   { attribute name : String;  attribute body : String;
                                      reference method : Goal.Method; } }
```

Views. Every agent owns a *view*, a metamodel (Definition **??**) that fixes what the agent may state. Views stay heterogeneous: each is sized to its agent's job, and how views correspond is recorded by trace models (Definition **??**) rather than by a shared schema, as in view-based development [**? ?**]. Metamodels of this kind are also used to define the views through which information about a system is retrieved [**?**] and the inputs of model-based analyses [**?**]. One view is special: the *goal view* MM_0, into which the lifter turns the task. The lifter separates examples from prose: every docstring example becomes an object of its own rather than a part of the docstring text. Whether a hand-off reads the examples is therefore visible in its data flow, which the coverage check of Section **??** relies on.

In the running example, the lifter parses the class skeleton of Listing **??** into a goal model M_0 with one Task, three Methods and three Examples (Listing **??**). Each method contains its own examples; the task additionally lists all examples in docstring order, which later lets a hand-off read the examples of other methods. A rule that reads only `Method.docstring` does not see the examples.

Hybrid rules. A hand-off is written as a set of matched rules (Definition **??**). Some of its bindings can be computed by a formula, such as copying a method's name; others, such as the body of a method, can only be written by an LLM. AgentHOT splits every rule along the line that ATL's two-phase execution already draws: matching, object creation and reference resolution stay with a deterministic engine, and only the bindings whose value no OCL expression can compute are delegated to an LLM.

Definition 8 (Hybrid rule, stochastic binding) A hybrid rule is a matched rule whose bindings are partitioned as $B_r = B_r^{\mathrm{str}} \uplus B_r^{\mathrm{sto}}$. A *structural* binding $f \leftarrow e \in B_r^{\mathrm{str}}$ is evaluated by the engine. A *stochastic* binding $b = f \leftarrow$ `@llm`$(\pi_b,\ e_b)$ `@check` $chk_b \in B_r^{\mathrm{sto}}$ targets a primitive attribute f; π_b is a prompt, e_b an OCL expression over the rule's source variables, and chk_b a validator. A *hybrid hand-off* is a transformation whose rules are hybrid rules.

Two design choices follow from the aim of making failures diagnosable. First, each LLM call sees a declared input, its *footprint*, rather than the whole conversation. What a value was produced from is then known exactly and can be recorded and replayed.

(A re-sample after a rejection additionally shows the LLM its own rejected value and the validator's message; Section **??**.) Second, a validator may read data that the LLM does not see; for example, a test validator may compare a test against the goal's examples even when the LLM writes the test from the docstring. A version stamp, finally, records the inputs a value was produced and checked from, so that the value can be recomputed when these inputs change. (A behaviour validator also sees the other values of the deliverable, which the stamp does not cover; Section **??** explains how the run handles this.)

Definition 9 (Footprint, validator reads, version stamp) Let b be a stochastic binding of rule r and m a match of r. A *location* is a pair (object, feature) of a source model. The *footprint* $fp_b(m)$ is the set of locations that the expression e_b navigates for m; they are the only values the LLM may see. The *validator reads* $vr_b(m)$ are the locations that chk_b inspects besides the candidate value; chk_b is a predicate over the candidate value and the values at $vr_b(m)$. The *version stamp* of an accepted value is the digest $\#(fp_b(m) \cup vr_b(m))$ of the values at all locations its production and its check read.

A value written by an LLM must not decide which objects exist; otherwise the structure of a hand-off would itself become random. We therefore require *stratification*: no guard and no structural binding may read a feature that a stochastic binding writes. This requirement is checked by W1 (Section **??**).

Our runtime departs from ATL in one respect. A reference binding whose expected type lies in a *source* view (for example `method <- m`, typed `Goal.Method`) creates a cross-view link to the source object instead of being resolved through the trace, because agents' views refer to the goal by design. ATL would resolve `m` to the target that `m`'s own rule created, which here has the wrong type; we apply that resolution only to references whose type lies in the *target* view. In addition, trace models are persisted, as in MeTAGeM-Trace [**?**].

We can now state what a hybrid hand-off means. Because its values are sampled, a hand-off cannot denote a single target model. It denotes the set of targets that its validators would accept.

Definition 10 (Semantics of a hybrid hand-off) Let the *structural part* T^{str} of a hybrid hand-off T be T with every stochastic binding removed. Given source models $\vec{M}$, the *skeleton* $T^{\mathrm{str}}(\vec{M})$ is computed by ATL's matching and initialisation, with the cross-view links above, and is deterministic. Like a relation of QVT Relations (QVT-R) [**? ?**], and unlike an ATL function, T denotes the *set* of admissible targets

$$[\![T]\!](\vec{M}) = \{M_t \mid M_t \sqsupseteq T^{\mathrm{str}}(\vec{M}) \ \wedge \ \forall (m, b) : \ chk_b(M_t[m, b], vr_b(m))\},$$

where $M_t \sqsupseteq T^{\mathrm{str}}(\vec{M})$ means that M_t extends the skeleton by exactly one value $M_t[m, b]$ for each match m and stochastic binding b. We require that T's target view is not one of its source views ($tgt(T) \notin src(T)$). An execution of T samples one element of this set, or *escalates* the bindings for which no admissible value was found. An escalated execution lies outside $[\![T]\!]$ and makes the run fail; an escalation is a recorded fault, not a hand-over to a person.

Listing 4 The three hybrid hand-offs of the admitted typed DevTeam. `@llm` marks a stochastic binding (second argument: footprint); `@check` its validator (Table **??**).

```
rule Method2Design {                              -- Goal → Design (owner: Architect)
  from m : Goal!Method
  to   d : Design!MethodDesign ( name ← m.name, method ← m,
         contract ← @llm('State a contract', Sequence{m.name, m.signature, m.docstring})
         @check nonempty(contract) ) }
rule Method2Test {                                -- Goal → Test (owner: Tester)
  from m : Goal!Method
  to   t : Test!TestCase ( name ← m.name, method ← m,       -- structural; cross-view link
         code ← @llm('Write unit tests for this method',
                     Sequence{m.signature, m.docstring, m.examples.call, m.examples.expected})
         @check test_valid(code, m) ) }                      -- behaviour validator
rule Design2Impl {                                -- Design, Test → Code (owner: Developer)
  from d : Design!MethodDesign, t : Test!TestCase (t.method = d.method)
  to   i : Code!MethodImpl ( name ← d.name, method ← d.method,
         body ← @llm('Implement the method',
                     Sequence{d.method.signature, d.contract, t.code})
         @check compiles(body) and passes_tests(body, t.code) ) }
```

A team of hand-offs denotes the set of tuples of models, one per view, in which every hand-off's target is admissible for its sources. On an acyclic hand-off graph, such tuples are obtained hand-off by hand-off in topological order. The footprint does not appear in $[\![T]\!]$: it constrains how a value is produced, not which values are admissible, and the runtime enforces it (Section **??**).

In the running example, Listing **??** shows the three hand-offs of the admitted DevTeam. In `Method2Test`, the engine decides which test cases exist and which method each one belongs to: one per goal method. The LLM writes only the code, from the method's signature, docstring and examples. No LLM can therefore forget to test `mark_done`. The binding `method <- m` is a cross-view link to the goal method, and the footprint of `code` contains the four listed paths.

3.3 What the builder proposes: typed teams

AgentHOT executes a team but does not design one. In AutoM2M, a builder LLM writes the views, rules and validators, so they become an artefact that must be checked. A free-form builder outputs role prompts (Listing **??**); an AutoM2M builder must output a *typed team*. Each component of a typed team makes explicit one aspect of the specification that a composition defect concerns: the views and hand-offs fix what is handed over (D2), write rights fix who owns which artefact (D3), goal obligations fix what must be covered (D1), the acceptance predicate fixes how completion is decided (D4), and the tool assignment fixes who can do what (D5).

Definition 11 (Typed team) A typed team is a tuple $\Theta = (A, V, \mathbf{T}, \omega, \kappa, G, \delta, \varphi)$ with the following components.

- A: a set of agents, each with a free-text role prompt.
- $V = \{MM_0, \ldots, MM_n\}$: a set of views with goal view MM_0.
- $\mathbf{T}$: a set of hybrid hand-offs, each with source views $src(T) \subseteq V$ and one target view $tgt(T) \neq MM_0$.
- $\omega : A \to 2^V$: the *write rights* of each agent.

Table 3 Validator library entries used in the examples and the evaluation, as instantiated for Python. Reads: arguments and paths the validator inspects besides the candidate value v.

Validator	Strength	Accepts the candidate value v if	Reads	Tools
`nonempty(v)`	form	v is a non-empty string	–	–
`compiles(v)`	form	v parses and compiles as Python	–	–
`smoke_test(v, b)`	behaviour	v equals the outcome of a fixed smoke test of the class assembled with body b	b	`exec`
`test_valid(v, m)`	behaviour	v runs, fails on a stub of m, and asserts every example of m	`m.signature`, `m.examples.*`	`exec`
`passes_tests(v, c)`	behaviour	the assembled class with v passes the test code c	c	`exec`
`examples_run(v, m)`	behaviour	the assembled class with v passes the examples of m	`m.examples.*`	`exec`

- $\kappa : A \to 2^{Tools}$: the tools of each agent.
- G: a set of *goal obligations* (C, s, μ, F_C). Each names a goal class C, a *scope* predicate s that selects the objects of C it concerns, a set of *anchor features* F_C of C (by default, all attributes of C), and a mode μ. Mode *checked* requires that the anchor features of every in-scope object reach a behavioural check; mode *delivered* requires a deliverable for every in-scope object.
- $\delta = (D, g)$: the *deliverable feature*, a target class D and its feature g whose values form the deliverable.
- φ: the proposed *acceptance predicate*, a conjunction of clauses that the engine evaluates at the end of a run (Definition **??**).

The goal view MM_0 and the goal model M_0 are produced by the lifter; the builder cannot change them.

The *owners* of a hand-off T are the agents whose write rights include its target view, $owner(T) = \{a \mid tgt(T) \in \omega(a)\}$; a stochastic binding's owners are those of its hand-off. In an admitted team, W2 makes the owner unique (Section **??**).

Restricted expressions and the validator library. To keep every admission rule decidable, a builder cannot write arbitrary OCL. Guards are conjunctions of comparisons between navigation paths and literals; structural bindings and footprints are navigation paths or literals; and validators are chosen from a library rather than written. Each library entry declares a *strength*, either *form* (the value parses, compiles or conforms) or *behaviour* (the value is executed against an oracle or a test); the tools it needs; and which of its arguments it *reads*. For a binding b, the read arguments determine the validator reads $vr_b(m)$, and the declared tools determine the tools $\tau(b)$ that the binding's owner needs; $\tau(b)$ is derived, not proposed. Table **??** lists the entries used in this paper.

In the running example, Listing **??** shows a plausible first proposal for DevTeam. It reads well: an Architect designs, a Developer implements, and a Tester runs the code. Yet each defect is present. No rule reads an Example, so E2.2 cannot influence anything that is checked (D1). The footprint of the Developer's binding refers to `d.returnType`, which the Design view does not declare (D2). Developer and Tester both write the

Listing 5 An illustrative first proposal of a typed DevTeam, in a concise textual syntax. Its views (not shown) differ from Listing **??** in that Test declares a class `TestRun` with an attribute `verdict`. It contains one instance of each defect D1–D5.

```
agents:   Architect{tools: []}  Tester{tools: []}  Developer{tools: [exec]}            -- D5
writes:   Architect: Design     Tester: Test       Developer: Code, Test                -- D3
goal:     (Example, all, checked, {call, expected})   (Method, all, delivered)
deliver:  MethodImpl.body
handoffs:
  Method2Design : Goal → Design
      contract ← @llm(.., Sequence{m.name, m.signature})        @check nonempty
  Design2Impl   : Design → Code
      body     ← @llm(.., Sequence{d.contract, d.returnType})   @check compiles         -- D2
  Impl2Run      : Code → Test
      verdict  ← @llm('Run and judge', i.body)          @check smoke_test(verdict, i.body)
done:     cover(G), valid, fresh, noEsc, Tester.says('ALL TESTS PASS')                  -- D4
```

Test view (D3). The acceptance predicate contains `Tester.says(...)`, a clause that holds when the Tester outputs the given text, that is, a claim (D4). And the Tester must run code without the `exec` tool (D5). (A builder that decodes under our JSON Schema cannot emit `Tester.says`, because done-clauses are an enumeration there; we keep the clause because it is the typical form of D4 in free-form teams.)

3.4 Running an admitted team

Given an admitted typed team, the runtime executes its hand-offs, records what every value was produced from, and decides at the end whether the run is done. Algorithm **??** shows both levels: RunTeam, which Algorithm **??** calls, and ExecuteHandOff, which executes one hand-off.

How to read Algorithm ??. RunTeam (lines 1–7) executes the hand-offs in topological order, which exists because W5 requires the hand-off graph to be acyclic, and repeats the pass until no model changes, because behaviour validators judge a value inside the deliverable assembled from all values accepted so far, so a sibling value accepted later can change an earlier verdict; the final clause *valid* re-checks every value. In ExecuteHandOff, line 9 performs ATL's matching and initialisation, made incremental [**?**]: it creates a target object for every new match, deletes the objects of matches that disappeared, and evaluates all structural bindings. This step is deterministic and calls no LLM. Lines 11–22 then visit every stochastic binding of every match, written (t, b) for the target object t and the binding b whose feature f it fills. Line 13 skips a value whose stamp equals the current digest of its footprint and validator reads: nothing it depends on has changed. Otherwise, the owner's LLM is prompted with the footprint (line 14), and a sample is accepted as soon as the validator accepts it (lines 18–19). After a rejection, the next prompt also contains the rejected value and the validator's message (line 21), so that the next sample differs. This feedback contains the LLM's own output and the validator's verdict; a verdict may mention the example that was checked, which is the only way in which validator reads reach the LLM. After k_{samp} rejections, the binding *escalates* (line 22): it is recorded as a failure, not handed to a person. The escalations of all hand-offs make the clause *noEsc* of φ fail and are the starting points of attribution.

Algorithm 2 AGENTHOT runtime: running a team and executing one hybrid hand-off

```
1  function RunTeam(Θ, M_0)
2      compile Θ into metamodels and rule modules; load M_0
3      repeat                                              ▷ at most p_max passes
4          for each hand-off T of Θ in topological order of the hand-off graph do
5              Esc_T ← ExecuteHandOff(T)
6      until no model changed in this pass
7      return (φ holds, the failed clauses and the values they name)        ▷ Def. ??

8  function ExecuteHandOff(T)        ▷ sources M⃗, target M_t, trace model TL_T
9      update the skeleton T^str(M⃗) in M_t and TL_T        ▷ match, create, delete, link
10     Esc ← ∅
11     for each trace link (m, t, r) ∈ TL_T and each stochastic binding b of r do
12         d ← #(fp_b(m) ∪ vr_b(m))                          ▷ digest of all that b depends on
13         if stamp(t, b) = d then continue
                                                            ▷ the value is fresh: keep it
14         prompt ← role prompt of owner(T), π_b, values at fp_b(m)
15         accepted ← false
16         for i = 1 to k_samp while not accepted do
17             v ← LLM_{owner(T)}(prompt)
18             if chk_b(v, values at vr_b(m)) then
19                 t.f ← v;  stamp(t, b) ← d;  accepted ← true
20             else
21                 append v and the validator's message to prompt          ▷ feedback
22         if not accepted then Esc ← Esc ∪ {(t, b)}
                                                            ▷ escalate
23     return Esc
```

Agents at run time. An agent is a role prompt, a model and a declared set of tools $\kappa(a)$. Agents do not converse. Each stochastic binding is one call to its owner's LLM, whose prompt consists of the role prompt, π_b and the footprint; producing a value uses no tools. Tools are needed only by validators, for example to execute code. Agents therefore see each other's work only through footprints, which is what makes a recorded footprint the input of a replay. The tool sets κ are a declared part of the specification, which W6 checks statically (Section **??**).

Incremental re-execution. When the team or its inputs change, only the affected values need to be produced again. Let a change Δ modify the locations $loc(\Delta)$. A pair (t, b) of a target object and a stochastic binding becomes *stale* if its footprint or validator reads contain a location in $loc(\Delta)$, or a location written by a pair that is already stale. The *stale set* $Stale(\Delta)$ is the least set of pairs, over the trace models of all hand-offs, that is closed under these two rules. All other values keep their stamps, so re-establishing the acceptance predicate costs at most $k_{samp} \cdot |Stale(\Delta)|$ LLM calls per pass. This is an analytical property of the runtime; we use it for repair (Section **??**) but do not evaluate arbitrary source changes in this paper.

In the running example, if E2.2 is changed to expect a `KeyError`, the stale set contains the test code of `mark_done` and, one step later, its implementation; all other values keep their stamps.

Deciding completion. Defect D4 arises when "done" is a claim. In AUTOM2M, the engine instead decides completion from the trace. To state that a goal object has been covered, we need a notion of what was derived from it, including objects that were only read through a footprint. Run records provide it.

Definition 12 (Run record, descendant) At run time, every trace link (m, t, r) carries, for each stochastic binding b of r, a *run record* $(b, fp_b(m), vr_b(m), stamp, v)$ with the accepted value v or an escalation mark. The link *connects* to each object in t every object of m and every object that owns a location in a recorded footprint or validator read. An object o' is a *descendant* of o if a chain of connections leads from o to o'.

Definition 13 (Engine clauses, acceptance) The *engine clauses* are evaluated on the final models and run records:

- *cover*(G) holds if every goal object o in the scope of an obligation (C, s, μ, F_C) has a descendant that holds an accepted value of a behaviour-validated binding (if $\mu = checked$) or of the deliverable feature (if $\mu = delivered$);
- *valid* holds if every accepted value passes its validator on the final models;
- *fresh* holds if every stamp equals the digest of the current values; and
- *noEsc* holds if no binding is escalated.

The acceptance predicate of an admitted team consists of all four engine clauses and, optionally, *library clauses* that the engine evaluates on the deliverable, such as running its public examples (W5). A run is *done* iff φ holds.

The clause *cover*(G) is deliberately one-sided: it asks only that a checked value descends from the goal object. That both the producer and the check of that value read the goal is ensured statically, by W4.

In the running example, each Example has as descendant the test case whose footprint read it, although no rule matches Example. Because no agent output can end the run, a Tester whose test for E2.2 fails cannot cause the team to be declared done.

3.5 Admission: six well-formedness rules

The checker decides whether a typed team may run. It evaluates six *admission rules* on the typed team alone, without task data, LLM calls or execution. W1–W3 adapt to LLM-authored teams the checks that static analysers perform on hand-written ATL transformations [**?**]; W4–W6 concern the team as a whole. Two graphs support them. The *hand-off graph* has an edge $MM_i \to MM_j$ for every hand-off with $MM_i \in src(T)$ and $MM_j = tgt(T)$. The second graph tracks data flow between features and is needed for coverage, as we explain first.

Why coverage needs feature-level data flow. The natural way to check coverage is at the level of classes: require a path in the hand-off rules from each goal class to

some behaviour-checked class. Such a *path-only* check is not enough. A builder can create one test case per method, written from the method's *name* only. A path from Method to a checked class then exists, yet no example ever influences a check. Coverage must therefore be checked at the level of features, and it must consider two roles that goal data can play: input to the producer of a value, and input to that value's check.

Definition 14 (Feature-level data flow) The data-flow graph $\mathcal{F}_\Theta$ has a node (C, f) for every class C and feature f of every view. A binding that writes feature g of class D contributes a *production* edge $(C, f) \to_p (D, g)$ for every (C, f) that its expression or footprint reads, and a *check* edge $(C, f) \to_c (D, g)$ for every (C, f) that its validator reads. Let $\mathcal{B}$ be the set of features written by stochastic bindings whose validator has behavioural strength.

A production edge transmits data: the producer sees it. A check edge transmits nothing; it only gates whether the value is accepted. For a goal to be really covered, the producer of a checked value must have seen the goal, *and* its check must test against the goal.

Definition 15 (Anchoring) Write $\to_p^*$ for a path of zero or more production edges. A goal obligation $(C, s, \mathit{checked}, F_C)$ is *anchored* at $(D, g) \in \mathcal{B}$ if, for every anchor feature $f \in F_C$,

(i) *production side:* $(C, f) \to_p^* \to_p (D, g)$, and

(ii) *check side:* $(C, f) \to_p^* (C', f') \to_c (D, g)$ for some node (C', f').

In the running example, the path `m.examples.call` in the footprint of `Method2Test` reads both `Method.examples` and `Example.call`, so it contributes production edges from both to `TestCase.code`; and `test_valid` reads `m.examples.*`, which contributes check edges. Figure **??** shows the resulting graphs for the admitted team and for the first proposal. The *one-sided* variant that requires only the check side (ii) is weaker: Section **??** shows that it admits teams whose producers never see the goal.

The six admission rules. Each rule below names the defect it targets, states what it requires and, where useful, notes its limits. D2 is split between two rules: W1 rules out its formal part (types and paths) and W3 its content part (what a target must contain).

W1 Hand-offs are well typed and stratified (against the formal part of D2). Every rule reads only its source views and the goal view, which every agent may read because agents' views refer to the goal by design, and writes only features that its target class declares. Every guard, binding and footprint path type-checks. Every stochastic binding targets a primitive attribute, has a non-empty footprint and uses a library validator with correctly typed arguments. Finally, the team is stratified (Section **??**).

W2 Every artefact has one writer (against D3). The write rights $\omega(a)$ are pairwise disjoint and together cover $V \setminus \{MM_0\}$, so $\mathit{owner}(T)$ is a single agent; and every class is created by at most one rule, so no object has two producers. This is stronger than ATL's requirement that each source tuple be matched by at most one rule. The rule is conservative: rules with provably disjoint guards could safely share a target class.

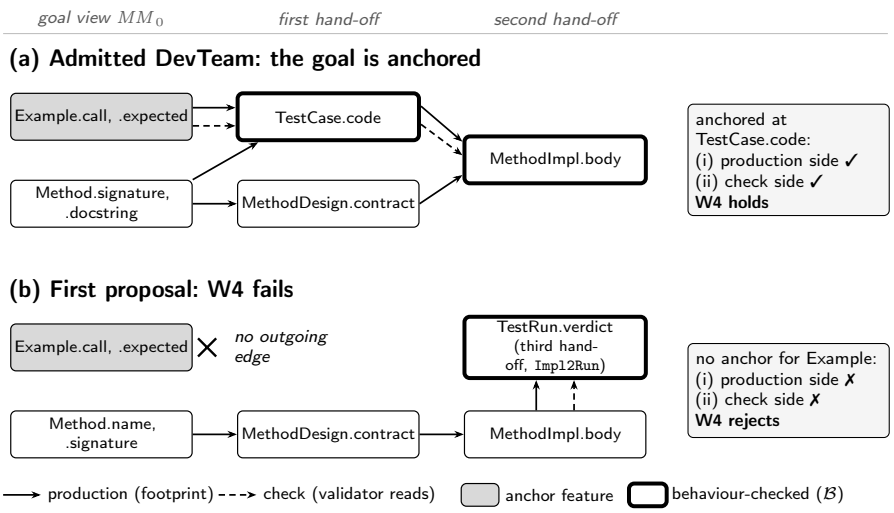


Figure 2 Feature-level data flow $\mathcal{F}_\Theta$ that W4 analyses, for the obligation (Example, *all*, *checked*, {*call*, *expected*}). Solid arrows are production edges (a footprint reads the feature), dashed arrows are check edges (a validator reads it). (a) In the admitted DevTeam, the examples reach TestCase.code both through the Tester's footprint and through its validator `test_valid`, so the obligation is anchored. (b) In the first proposal (Listing **??**), no edge leaves the examples, so W4 rejects the team although it does test *something*. Edges from Method.signature to MethodImpl.body are omitted for readability.

W3 Targets are complete (against the content part of D2). Every mandatory feature of a created class is bound exactly once. Every reference binding whose type lies in the target view is *resolvable*: some rule creates an object of the expected class from the class of the source object that the binding yields. Every reference binding whose type lies in a source view yields objects of that type.

W4 Every goal is anchored (against D1). W4 has four parts.

(a) Every obligation of mode *checked* is anchored at some feature of $\mathcal{B}$ (Definition **??**).

(b) Every source-pattern class of a rule on an anchoring path should be *total*: it is a goal class, or it is created by a rule whose guard is implied by the obligation's scope. Totality ensures that the rule's other source variables can be filled for every in-scope goal object. Because implication between guards is undecidable in general, the checker tests it syntactically (the guard is `true`, equals the scope, or is a join) and *warns* about every other guard instead of rejecting the team.

(c) Every obligation of mode *delivered* reaches the deliverable feature δ: at least one of its anchor features has a path of production edges to δ.

(d) Every view is reachable from MM_0 in the hand-off graph and reaches a view that contains a feature of $\mathcal{B}$, so no agent is idle and no agent's work goes unverified.

W5 The engine decides completion (against D4). φ contains all four engine clauses of Definition **??** and, apart from them, only library clauses; every clause is evaluated by the engine, and no clause may be an agent's claim. In addition, the hand-off graph is acyclic, so that hand-offs can be executed in topological order (Definition **??**).

Algorithm 3 ADMIT: deciding whether a typed team Θ may run

Require: typed team $\Theta = (A, V, \mathbf{T}, \omega, \kappa, G, \delta, \varphi)$; validator library
Ensure: the set *diag* of diagnostics; Θ is admitted iff $diag = \emptyset$

1 $diag \leftarrow \textsc{TypeErrors}(\mathbf{T}, V) \cup \textsc{StratificationErrors}(\mathbf{T})$ ▷ *W1*
2 $diag \leftarrow diag \cup \textsc{SharedWriters}(\omega, \mathbf{T})$ ▷ *W2*
3 $diag \leftarrow diag \cup \textsc{IncompleteTargets}(\mathbf{T})$ ▷ *W3*
4 $(\mathcal{F}_\Theta, \mathcal{B}) \leftarrow$ feature-level data-flow graph and behaviour-checked features ▷ *Def.* **??**
5 **for** each obligation $o = (C, s, \mu, F_C) \in G$ **do** ▷ *W4(a)–(c)*
6 **if** $\mu =$ *checked* **and** no $(D, g) \in \mathcal{B}$ satisfies $\textsc{Anchored}(o, (D, g))$ **then**
7 $diag \leftarrow diag \cup \{(\mathsf{W4}, o, \text{“not anchored”})\}$
8 **if** $\mu =$ *delivered* **and** no $f \in F_C$ has a production path to δ **then**
9 $diag \leftarrow diag \cup \{(\mathsf{W4}, o, \text{“does not reach the deliverable”})\}$
10 warn about every guard on an anchoring path of o that is not implied by s
11 **for** each view $V' \in V$ **do** ▷ *W4(d)*
12 **if** V' is unreachable from MM_0 **or** reaches no feature of $\mathcal{B}$ **then**
13 $diag \leftarrow diag \cup \{(\mathsf{W4}, V', \text{“idle or unverified view”})\}$
14 $diag \leftarrow diag \cup \textsc{NonEngineClauses}(\varphi) \cup \textsc{Cycles}(\mathbf{T})$ ▷ *W5*
15 **for** each stochastic binding b and each writer a of its target view **do** ▷ *W6*
16 **if** $\tau(b) \not\subseteq \kappa(a)$ **then** $diag \leftarrow diag \cup \{(\mathsf{W6}, b, a)\}$
17 **return** *diag*

W6 Work goes to agents that can do it (against D5). For every stochastic binding b, $\tau(b) \subseteq \kappa(a)$ for its owner a, where $\tau(b)$ comprises the tools that b's validators need and any tools the binding declares. The rule mirrors Gaia's permissions, under which a role cannot act outside what it is granted [**?**]: an agent that is not declared to execute code must not own a value whose acceptance depends on executing code. When W2 fails and a view has several writers, W6 is evaluated for each of them.

How to read Algorithm ??. The checker evaluates all six rules and never stops at the first violation, so that a single call reports every problem of a proposal. Lines 1–3 are local checks of rules, write rights and target classes. Line 4 builds the data-flow graph of Definition **??**, on which the coverage checks of lines 5–13 operate. For an obligation of mode *checked*, line 6 searches for a behaviour-checked feature at which it is anchored, that is, which every anchor feature reaches both through production edges and through a check edge (Definition **??**; Figure **??**); this is a reachability question on $\mathcal{F}_\Theta$. Line 10 records the guards that the checker cannot prove harmless as warnings. Lines 11–13 reject idle views and views whose work is never verified. Line 14 checks completion and acyclicity, and lines 15–16 compare the tools a binding needs with those of every agent that may write its target. Each diagnostic names the violated rule, the offending element (a rule, a path, a write right or a clause) and a hint. Unlike an LLM critic's comments, the diagnostics report *every* violation of the admission rules, and they point to a concrete element rather than advising the builder to “improve coordination”.

In the running example, Listing **??** shows the output of our checker on the first proposal (Listing **??**): one violation per seeded defect and no other.

Listing 6 Checker output on Listing **??**.

```
W1  Design2Impl.body: footprint path 'd.returnType':
        Design!MethodDesign has no feature 'returnType'
W2  view Test written by ['Developer', 'Tester']; needs exactly one writer
W4  goal (Example, all, checked): anchor features {call, expected}
        reach no behaviour-checked value
W5  done-clause Tester.says('ALL TESTS PASS') is not an engine clause
W6  Impl2Run.verdict needs ['exec']; writer Tester has []
REJECTED: 5 violation(s)
```

Properties. The checker runs on every candidate inside the builder's loop (Section **??**), so its cost matters.

Proposition 1 (Cost) *W1–W6 are decidable on Θ in time polynomial in $|\Theta|$ (classes, features, rules, bindings and path segments).*

Sketch W1 navigates each path once; W2, W3 and W6 compare finite sets. W4(a) and (c) are reachability problems over $\mathcal{F}_\Theta$, whose size has at most one node per feature and one edge per path segment of each binding, so a breadth-first search from each anchor feature costs $O(|\mathcal{F}_\Theta|)$; implication between guards and scopes is tested syntactically, in constant time per guard. W4(d) and the cycle test of W5 are searches on the hand-off graph. □

A second property explains why admission makes failures easier to locate. We would like a failed *cover*(G) clause never to stem from missing structure, so that it always points at a value an LLM was asked to produce. The property needs one more notion. A guard that equates two structural references to the same goal object is a *join*; in the running example, `t.method = d.method` in `Design2Impl` is a join.

Proposition 2 (Structural coverage) *Let Θ be admitted, and let every guard of the rules on the paths that anchor an obligation $(C, s, \mathit{checked}, F_C)$ either be implied by s or be a join. Then, for every object o of class C that satisfies s, the skeletons T^{str} create a chain of descendants of o that ends in a target object t with a behaviour-checked feature, whose footprint and validator reads both contain data derived from o. Consequently, if cover(G) fails for o at the end of a run, some stochastic binding on that chain is escalated or invalid.*

Sketch By W1 (stratification), skeletons do not depend on LLM-written values, so matches depend only on goal data and on objects created by earlier skeletons. We proceed by induction along an anchoring path of Definition **??**. *Hypothesis:* after the skeletons of the first i rules on the path have run, an object of the class of the i-th node exists and is a descendant of o. *Step:* the $(i{+}1)$-th edge stems from a rule whose source pattern contains that object, or an object from which the rule's footprint navigates to it. (1) This source variable is filled by the hypothesis. (2) The other source variables are filled because their classes are total (W4(b)). (3) The rule's guard holds, either because s implies it or because it is a join whose two sides were linked to the same goal object by structural bindings. (4) The rule therefore fires and, by W3, creates its target with all mandatory features. Hence every object on the chain exists, and only stochastic values can be missing. By Algorithm **??**, a missing or rejected value is escalated, or reported by *valid*. □

Algorithm 4 BUILD: proposing and revising a typed team under the checker

Require: task x; goal model M_0; budgets k_{prop} (proposals), k_{rev} (revision samples per round), k_{adm} (rounds)
Ensure: a team Θ and its diagnostics *diag* (*diag* $= \emptyset$ iff Θ is admitted)

▷ *Phase 1 (steps A and B): sample candidates and let the checker rank them*
$\mathcal{C} \leftarrow \emptyset$
for $i = 1$ **to** k_{prop} **do**
  $\Theta_i \leftarrow \textsc{Propose}(x, M_0)$ ▷ *decoded under the typed-team schema*
  $\mathcal{C} \leftarrow \mathcal{C} \cup \{(\Theta_i, \textsc{Admit}(\Theta_i))\}$
  if $\textsc{Admit}(\Theta_i) = \emptyset$ **then break** ▷ *admitted: stop sampling*
$(\Theta, \mathit{diag}) \leftarrow$ the candidate in $\mathcal{C}$ with the fewest diagnostics
▷ *Phase 2 (step C): revise only the parts that the diagnostics name*
for *round* $= 1$ **to** k_{adm} **while** *diag* $\neq \emptyset$ **do**
  $P \leftarrow \textsc{Fragments}(\Theta, \mathit{diag})$ ▷ *rules, views, write rights, ... that diag names*
  $\Pi \leftarrow \textsc{TypedPaths}(\Theta, P)$ ▷ *paths that type-check from each rule's variables*
  $\mathcal{R} \leftarrow \{\, \Theta \lhd \textsc{ReviseFragments}(\Theta, \mathit{diag}, P, \Pi) \mid \text{one call per } i = 1..k_{rev} \,\}$
  if no $\Theta' \in \mathcal{R}$ has fewer diagnostics than Θ **then**
    $\mathcal{R} \leftarrow \mathcal{R} \cup \{\textsc{ReviseWhole}(\Theta, \mathit{diag})\}$ ▷ *fallback: one whole-team revision*
  $\Theta^* \leftarrow$ the team in $\mathcal{R}$ with the fewest diagnostics
  if $|\textsc{Admit}(\Theta^*)| \leq |\mathit{diag}|$ **then** ▷ *never accept a worse team*
    $(\Theta, \mathit{diag}) \leftarrow (\Theta^*, \textsc{Admit}(\Theta^*))$
return (Θ, diag)

Proposition **??** turns locating the symptom of a coverage failure into a lookup: such a failure is never a silent structural gap but points at a named binding. Its premise on guards is exactly what the checker verifies syntactically in W4(b); for a team admitted with a guard warning, the proposition does not apply, and *cover*(G) still detects the gap at run time, although without naming a binding.

Limits. Whether a guard excludes in-scope objects is undecidable in general. The checker accepts guards that are implied by the obligation's scope, warns about all others, and *cover*(G) enforces coverage at run time. Coverage is also relative to what is declared: an obligation that the builder does not declare is not checked (Section **??** shows the consequence for deliverables). Admission does *not* guarantee that the views capture what the task needs, that validators are strong enough, or that agents reason well.

3.6 Building a team under the checker

The builder is the only component that designs, and it is an LLM. Its proposals are imperfect: in our study, the most frequent first violation of a rejected proposal is a reference to a feature that the builder never declared (Section **??**). BUILD (Algorithm **??**) therefore treats the checker as an oracle that the LLM consults, but never overrules: the LLM proposes and revises, and only Algorithm **??** admits.

How to read Algorithm ??. Phase 1 (lines 1–7) samples up to k_{prop} proposals. Every proposal is decoded under the JSON Schema of the typed-team format, so it always has the *shape* of a typed team; whether it is well typed, complete and anchored remains the checker's decision. Because a check takes milliseconds (Section **??**), the checker can rank the candidates by their number of diagnostics (line 7). Phase 2 (lines 8–17) revises the best candidate for at most k_{adm} rounds. Line 10 maps each diagnostic to the *fragments* of the team it concerns: a rule together with the classes it reads and creates and the sources of its hand-off, a view, the write rights ω, the tools κ, the obligations G and the deliverable δ, or φ; for an unanchored goal, the fragments include the rules and views that should carry it. Line 11 lists, for each diagnosed rule, short navigation paths that type-check from its source variables, which addresses the most frequent error, an undeclared feature. Line 12 asks the LLM k_{rev} times for replacements of these fragments only, and the operator $\lhd$ puts them back into Θ deterministically (it is the special case of $\Theta \oplus \Delta$ in which the delta replaces named fragments), so a part that was not diagnosed is never regenerated. Only if no localised revision reduces the number of violations is one whole-team revision tried (lines 13–14). A revision is accepted only if it is not worse than the current team (lines 16–17). We designed the localised revision because, in pilots, small builders often re-introduced an earlier violation when they rewrote the whole team. A small ablation does not confirm that it helps: for Qwen2.5-Coder 7B on 40 ClassEval tasks (one seed), a builder that makes one proposal and revises the whole team admitted slightly more sessions than Algorithm **??** (28 against 21; 14 against 7 discordant tasks, exact McNemar $p = .19$).

3.7 Attribution and checked repair

Transcript-based methods identify the responsible agent of a failure in about half of the cases and the decisive step in about one in seven (Section **??**). AutoM2M separates two questions. *Where* does a failure show? This is the *symptom* binding, and the trace answers it by lookup. *Why* did it fail? This is the *cause*, which may lie in another binding, and a bounded replay answers it.

Locate. If φ fails, the engine knows which clauses fail and, through the trace models, where. A failed *valid* clause or an escalation names one pair (t, b): a target object and a stochastic binding, and with them its rule, hand-off and owner, and the stamped footprint its LLM saw. A failed *cover*(G) clause names, by Proposition **??**, a binding on the anchoring chain. Locating the symptom is therefore a lookup, not an inference over a transcript. Attribution needs the input of each step [**?**]; in AgentHOT, that input is a declared, bounded footprint.

Classify. The symptom binding is not always the cause. We distinguish five fault classes, each with its remedy:

- Validator: the check itself is unreliable. Remedy: replace the validator (builder).
- Sampling: the composition is adequate, but the owner's LLM was unlucky or too weak. Remedy: retry, or use a stronger model.
- Footprint(j): the LLM did not see information it needed, an instance of D2. Remedy: widen the footprint by the paths, of at most j references, that make replays pass.

Algorithm 5 ATTRIBUTE: classifying the fault behind a failed value (t, b)

Require: failed value v of binding b of rule r in match m; recorded footprint $Fp = fp_b(m)$; replay cap n_{rep}; hop limit h;
acceptance test $ok(\cdot)$, by default $ok(u) = chk_b(u, \text{values at } vr_b(m))$
Ensure: a fault class, with the information its repair needs

1 **if** re-running $ok(v)$ three times does not reproduce the recorded verdict **then**
2 **return** VALIDATOR ▷ *the check is unstable; no LLM call*
3 **if** REPLAYS(b, Fp) **then return** SAMPLING ▷ *the same input can succeed*
4 **for** $j = 1$ **to** h **do**
5 $Fp_j \leftarrow Fp \cup$ PATHSWITHIN(r, j) ▷ *paths of at most j references*
6 **if** REPLAYS(b, Fp_j) **then return** FOOTPRINT(j) with the minimal added paths
7 **for** each value u in Fp or $vr_b(m)$ that a stochastic binding b' wrote **do**
8 $u' \leftarrow$ re-sample b' until b''s own validator accepts, at most n_{rep} times
9 **if** u' exists **and** REPLAYS$(b, Fp[u \mapsto u'])$ **then**
10 *inner* $\leftarrow$ ATTRIBUTE(b' with failed value u, ok') where $ok'(w)$ holds iff b''s validator accepts w **and** b passes with w in place of u
11 **return** UPSTREAM(b') with *inner* ▷ *the cause lies one hand-off earlier*
12 **return** SPECIFICATION ▷ *no supported value satisfies the check*

13 **function** REPLAYS(b, Ψ) ▷ *adaptive: stop at the first pass*
14 **for** $i = 1$ **to** n_{rep} **do**
15 **if** $ok(\text{LLM}_{owner(b)}(\pi_b, \text{values at } \Psi))$ **then return** *true*
16 **return** *false*

- UPSTREAM(b'): an input of the binding, produced by another stochastic binding b', is wrong. Remedy: re-sample b', after attributing it in turn.
- SPECIFICATION: no value that the footprint supports satisfies the prompt and the validator. Remedy: a builder delta.

Algorithm **??** tests these classes in a fixed order, from the cheapest test to the most expensive one, and SPECIFICATION is the default when no test succeeds.

How to read Algorithm ??. Lines 1–2 test the validator without calling an LLM: the failed value is checked again n_{val} times ($n_{val} = 3$ in our evaluation), and if the verdict changes, the check itself is unreliable. Line 3 replays the binding on its recorded footprint Fp: if some replay passes, the composition was adequate and the owner's LLM was unlucky or weak (SAMPLING). Lines 4–6 widen the footprint by every path from the rule r's source variables that follows at most j references and ends in an attribute; if a replay then passes, the LLM lacked information (FOOTPRINT), and the paths that are needed are kept for the repair (a greedy minimisation drops each added path whose removal still lets a replay pass). Lines 7–11 test the inputs that other LLMs produced: each upstream value u is replaced by a fresh value u' that passes its own validator, and if b then passes, the upstream value was the cause. The recursive call of line 10 classifies why u was wrong; there, a candidate w for u counts as passing

only if it passes b''s own validator *and* the downstream binding b then passes, first with its old failed value and otherwise with one fresh sample. This criterion ties the upstream diagnosis to the failure that was observed. Line 12 is the default: no value that the footprint supports satisfies the prompt and the validator, so the specification itself must change. REPLAYS (lines 13–16) stops at the first passing sample; we call this the *adaptive* replay budget.

Two cases fall outside the five classes. A failed clause that names no value, such as a failed library clause, is reported as UNLOCATED; a coverage failure on an object for which no binding exists is reported as COVERAGE. Both are passed to the builder, like a specification fault. They did not occur among the injected faults of Section **??**.

Remark 1 (Cost of attribution) For one binding b, Algorithm **??** makes at most n_{rep} replays for the sampling test, $h \cdot n_{rep}$ for the footprint tests, and, for each of the $|up(b)|$ upstream values, at most n_{rep} re-samples and n_{rep} replays, where $up(b)$ is the set of upstream values in b's footprint and validator reads: in total at most $(h + 1 + 2|up(b)|)\, n_{rep}$ LLM calls, plus the replays that minimise a footprint repair. Recursion visits at most as many bindings as the longest path in the hand-off graph.

Checked repair. The planner of Algorithm **??** (line 10) chooses the cheapest repair for each fault report. Sampling and upstream faults are repaired mechanically, by a retry of b or a re-sample of b'; these repairs change values but not the team, so they need no check. A footprint fault becomes an automatic delta that widens the footprint by the paths that attribution found. Validator, specification, coverage and unlocated faults require a delta proposed by the builder's LLM, which rewrites only the fragments that the fault reports name, as in Algorithm **??**. $\Theta \oplus \Delta$ denotes the typed team with Δ's edits applied. The checker evaluates it with Algorithm **??**, and APPLY installs it in the running team only if it is admitted. As with consistency-preserving operators [**? ?**], deltas differ in what they preserve, and APPLY uses one of three modes:

- *in place*, for a delta that changes only rules, prompts, footprints or validators: the affected values lose their stamps and become stale, and all other accepted values are kept;
- *by extension*, for a delta that only adds views, agents or hand-offs: the compiler generates the new rule modules, so that new agents receive work for everything already built;
- *rebuild*, for any other delta, as a free-form builder must always do.

Changes flow only from the team model to the runtime, never back: the runtime is regenerated from the team model.

The running example, end to end. We follow `TaskBoard` from the first proposal to an accepted deliverable.

1. *Admission.* The first proposal (Listing **??**) is rejected with five diagnostics (Listing **??**). The reviser rewrites only the named fragments: the hand-offs and views for W1 and W4 (including the rules that should carry the unanchored goal), the write rights for W2, φ for W5, and the agents' tools for W6. Its revision replaces

`Impl2Run` by `Method2Test` (Listing **??**), gives the Tester `exec`, removes the Developer's write right on Test, adds `m.docstring` to the footprint of `Method2Design`, replaces `d.returnType` by `d.method.signature`, lets `Design2Impl` read the Test view (joined on the method) with the tests in its footprint and a `passes_tests` validator, and drops the Tester's claim from φ. The revised team is admitted.

2. *Run.* The implementation of `mark_done` escalates after three samples fail `passes_tests`.
3. *Locate.* Lookup names the pair $(impl_{\texttt{mark_done}}, \texttt{body})$, owned by the Developer.
4. *Classify the implementation.* Replays on the same footprint fail, and widening the footprint does not help. The upstream test, however, is suspicious: the Tester's first test called `mark_done(1)` on a fresh board, without the `add_task` that E2.1 relies on, so every correct implementation failed it. `test_valid` had accepted that test because it runs, fails on a stub and asserts every example. Of up to three re-samples of the test from its own footprint, one happens to create a board first; with this test, the replays of the implementation pass. The fault is UPSTREAM, in $(test_{\texttt{mark_done}}, \texttt{code})$, owned by the Tester.
5. *Classify the test.* Algorithm **??** recurses on the test, with the criterion that a candidate test must also let the implementation pass. This time, none of the three replays of the test on its own footprint adds the missing setup, so the sampling test fails: the setup is not part of what the Tester sees, and only an occasional sample guesses it. Widening the footprint by two references adds `m.task.examples.*`, which contains E1.1, and every replay then passes: a FOOTPRINT(2) fault of `Method2Test`. The two steps also show why sampling and footprint faults can be hard to separate: a missing input that an LLM can sometimes guess makes a footprint fault look like bad luck.
6. *Repair.* The automatic delta widens that footprint, is admitted, and is applied in place. Only the test and the implementation of `mark_done` become stale and are re-sampled, after which φ holds.

A stronger `test_valid` that also checked the order of examples would have rejected the first test.

3.8 Scope and limitations

AUTOM2M checks that a team fits *together*, not that its views capture the task or that its validators are strong. Hand-off graphs are acyclic, LLMs fill in attributes but do not create structure, and teams whose value lies in open-ended dialogue do not decompose naturally into views and rules.

4 Related Work

We organize related work into seven lines. For each, we state what it contributes and where AUTOM2M differs; Table **??** summarizes the comparison on the properties that matter for composition.

4.1 Hand-designed LLM multi-agent teams for software engineering

Role-based teams emulate human development processes. MetaGPT gives roles structured output documents [**?**]; ChatDev chains role pairs through development phases [**?**]; Self-Collaboration and MapCoder split analysis, coding, testing and debugging [**? ?**]; and AutoGen and AgentVerse let developers wire arbitrary topologies [**? ?**]. Two surveys list coordination and verification among the open problems [**? ?**]. These teams have a human architect. Beyond the document types of MetaGPT, nothing checks before execution that every requirement reaches a test, that each artefact has one writer, or that the system rather than an agent decides completion. AutoM2M provides these properties for teams that no human reviews.

4.2 Automatic team builders and learned orchestration

Per-task builders design a team for each task. CaptainAgent [**?**], AutoAgents [**?**] and MAS-Zero [**?**] rely on reflector, observer or meta-feedback LLMs; EvoAgent evolves agents [**?**]; DyLAN selects them [**?**]; and MetaAgent represents the team as a finite state machine that makes control explicit [**?**]. *Offline builders* search for agentic workflows as code or graphs: ADAS [**?**], AFlow [**?**], GPTSwarm [**?**], MaAS [**?**] and the GNN-based G-Designer [**?**]. *Learned orchestrators* are trained with reinforcement learning to pick the next agent (Puppeteer [**?**]), to write natural-language workflows (Conductor [**?**]) or to emit whole teams (MAS-Orchestra [**?**]); other work evolves agent graphs or refines teams from textual feedback [**? ? ?**].

All of these approaches optimise *which* team to build against an end-to-end score or another LLM's judgement. Code-based workflows are parsed and executed, and MetaAgent checks its state machine for reachability. None, however, states as a checkable interface what is handed over between each pair of agents, and none checks data-flow coverage or the ownership of hand-offs before the team runs. AutoM2M is complementary: it proposes no new search strategy but a target representation that any builder can emit, and a check that a builder can run inside its loop in milliseconds, with diagnostics that name a rule, path or write right.

4.3 Failure analysis and attribution for multi-agent systems

MAST derives 14 failure modes from more than 1,600 annotated traces [**?**], and AgentAsk locates errors on hand-off edges [**?**]. Further studies analyse why agent frameworks fail, how team structure must fit the task, and why communication does not imply coordination [**? ? ? ? ? ?**]. Who&When and TraceElephant attribute failures to agents and steps from transcripts [**? ?**]. These works characterise failures after the fact and from prose. AutoM2M prevents a class of them before execution and, when a typed team fails, locates the symptom by lookup and classifies it by bounded replay. Our defect analysis (Section **??**) builds on their taxonomies and data, and RQ2 uses their attribution methods as baselines.

4.4 Structured communication and typed LLM programming

Agent communication languages gave messages explicit performatives and shared ontologies long before LLMs [**?**]. For LLM agents, MCP and A2A standardise the transport between agents and tools [**? ?**]; Agora lets agents negotiate protocol documents [**?**]; and PatchBoard grounds collaboration in schema-validated patches to a shared state [**?**]. Constrained decoding guarantees that outputs conform to a grammar or JSON schema [**?**]. Practitioner frameworks type hand-offs too: LangGraph declares a typed state schema for a graph of agents [**?**], CrewAI validates a task's output against a Pydantic model [**?**], and AgentSpec enforces user-written rules on agent actions at run time [**?**].

These approaches make an individual *message* or action well formed, reliably transported or permitted. They do not check properties of the *assembled team*: that every obligation reaches a check that reads it, that every artefact has one writer, that completion is decided by the system, or that work goes to an agent with the needed tools. A single shared schema also removes heterogeneity by decree, whereas AutoM2M keeps one view per agent, sized to its job, and records correspondences in trace models. We use a PatchBoard-style shared schema as a baseline (Section **??**).

Closest to the AgentHOT runtime are LLM programming systems that interleave deterministic control with generated parts: LMQL constrains and scripts generation inside a query language [**?**], SGLang executes structured LM programs efficiently [**?**], and DSPy composes LM modules whose inputs and outputs are typed by signatures [**?**]. These systems control *how* one program calls an LLM. DSPy's signatures and assertions come closest to our hand-offs: they type the inputs and outputs of one module and can reject an output. What a signature does not fix is *how many* values exist and *which* goal objects they belong to. AgentHOT fixes both by model-transformation rules: a matched rule creates one target per goal method, so no LLM can forget to test `mark_done`, and a trace link records the match behind each value. These are the properties on which the feature-level coverage check (W4) and attribution by lookup rely, together with three more that AgentHOT adds: each value has a declared footprint, is accepted only behind a validator and is stamped for incremental re-sampling, and the engine, not an agent, decides completion.

4.5 Static analysis, verification and testing of model transformations

MDE has a mature body of work on checking individual transformations. Static analysers for ATL detect typing errors, unresolved bindings, rule conflicts and missing mandatory features [**?**]; invariant-based and model-finding approaches verify declarative transformations against contracts [**? ?**]; mutation analysis assesses transformation test suites [**?**]; and a survey covers testing and debugging [**?**]. Fault localisation ranks suspicious rules statically or from test spectra [**? ?**], and anATLyzer proposes speculatively checked quick fixes [**?**]. Transformation intents make the properties a transformation must satisfy explicit [**?**]. W1–W3 are deliberately close to established static checks [**?**]. Our diagnostics play the role of anATLyzer's quick-fix hints, but the fixes are proposed by an LLM and re-checked; and unlike spectrum-based localisation [**?**

], which ranks rules by test outcomes, attribution in AutoM2M replays one binding on its recorded footprint. We adopt mutation analysis [**?**] to evaluate the checker itself.

Closer to the *team* level, transformation chains have been typed and checked for the compatibility of their connected metamodels [**?**], and chains between incompatible metamodels have been assembled automatically [**?**]; footprinting estimates which parts of a model an operation reads [**?**]. This work assumes that a person writes the transformations and that their execution is deterministic. Our footprints are *declared* rather than estimated. Chain typing asks whether a chain's metamodels are compatible; anchored coverage (W4) asks a finer question, namely whether specific *goal features* flow into both the production and the check of a value. This question matters most when values are produced by a stochastic author and accepted by validators, because then a value can pass its check without its producer ever having seen the goal. Engine-decided completion (W5) and capability fit (W6) concern the team rather than any single transformation.

4.6 Large language models in model-driven engineering

LLMs have been evaluated for domain modelling [**?**], model completion [**?**] and UML modelling tasks [**?**], with useful but imperfect accuracy; an evaluation of LLM-based domain-modelling assistance reaches a similar conclusion [**?**]. Prompting itself can be model driven [**?**], LLMs have completed software model evolution from edit histories [**?**], and mapping studies chart LLM use across MDE activities and identify LLM-written metamodels and transformations as an open setting [**? ?**]. Recent studies ask LLMs to write transformations in model transformation languages and report that syntactic and semantic errors remain frequent [**? ?**]. This line evaluates *individual* LLM-generated models or transformations. AutoM2M accepts that a proposal will be imperfect and checks how the generated views and transformations fit together; RQ3 measures how often an LLM reaches an admissible team. Where an LLM performs a model transformation, it replaces the transformation as a whole [**?**]; we are not aware of MDE work in which rules delegate individual bindings to an LLM behind validators, as AgentHOT does.

4.7 Software architecture and agent-oriented software engineering

Architectural mismatch, design by contract and specified connectors make the assumptions between components explicit and checkable [**? ? ? ?**], and Gaia gave every agent role responsibilities, permissions and protocols [**?**]. AutoM2M transfers these remedies to systems that an LLM composes without review: write rights play the role of Gaia's permissions, hand-offs that of connectors, and φ that of a system-level postcondition.

5 Evaluation

5.1 Research questions

AutoM2M changes how a team is built and run: the builder must write a typed team, a checker decides whether the team may run, and a trace records how every value was

Table 4 Guarantees before execution and support for attribution (our reading of each approach as published, without extensions that a developer could add by hand; ✓ provided, ❍ partly or by convention, ✗ not provided, – not applicable). *LLM-built*: the team is composed by an LLM. *Msg. form*: individual hand-offs are schema-checked. D1–D5: the defect is ruled out before the team runs. *Attrib.*: a failure can be traced to a responsible component without reading a transcript. *Dial.*: agents may converse freely, which AutoM2M gives up. For AutoM2M: D1 only relative to the declared obligations; D5 only for tools in the declared vocabulary.

Approach	LLM-built	Msg. form	D1	D2	D3	D4	D5	Attrib.	Dial.
Hand-designed teams [? ?]	✗	❍	✗	❍	❍	✗	✗	✗	✓
Typed agent frameworks [? ?]	✗	✓	✗	❍	✗	✗	✗	✗	✓
Per-task builders [? ?]	✓	✗	✗	✗	✗	✗	✗	✗	✓
FSM builder (MetaAgent) [?]	✓	✗	✗	✗	❍	❍	✗	✗	✓
Workflow search [? ?]	✓	❍	✗	✗	✗	✗	✗	✗	✓
Learned orchestration [?]	✓	❍	✗	✗	✗	✗	✗	✗	✓
Structured messages [? ?]	✗	✓	✗	❍	✗	✗	✗	❍	❍
LLM programming [? ?]	✗	✓	✗	❍	✗	✗	✗	✗	✓
MT static analysis [? ?]	✗	–	✗	✓	❍	–	–	❍	–
Typed MT chains [?]	✗	✓	✗	❍	–	–	–	✗	–
AutoM2M (this paper)	✓	✓	❍	✓	✓	✓	❍	✓	✗

produced. Such machinery is not free. It costs tokens and time, and a builder that cannot write an admissible team produces no team at all. The evaluation therefore asks what an engineer *gets* for this price. We evaluate the two benefits that the design promises, and the mechanism that delivers both. For each, we report the price next to the benefit.

RQ1 *(Reliability): When an* AutoM2M *team reports that it is done, can we believe it, and when it fails, does it say so?*
Every condition in our study has its own way of ending a run: an agent stops, a team sends `TERMINATE`, or the engine finds the acceptance predicate φ true (Table **??**, last column). A developer who uses a team without re-checking every output relies on this signal. A failure that the team reports as a success is the costly case, because nobody looks at it. We call it a *silent failure*. RQ1 compares, over all 44,352 runs, how often each condition fails silently, how often a failed run is flagged as unfinished, and how much a "done" raises the chance that the result is correct. *Price measured:* task success, how often a run reports "done" at all, and tokens.

RQ2 *(Traceability): When an* AutoM2M *team fails, can the failure be traced to the responsible part of the team, explained and repaired there?*
In a free-form team, a failure leaves a chat transcript, and finding the responsible agent means reading it. In AutoM2M, every value has a trace link to the rule, the agent and the inputs that produced it (Section **??**). RQ2 injects known faults into running teams and asks whether trace-based attribution finds the responsible binding and names the kind of fault more accurately than four transcript-based methods from the literature. It also asks whether a checked repair of the diagnosed part works as well as rebuilding the whole team. *Price measured:* LLM calls and time spent on attribution.

RQ3 *(Prevention): Does the admission check catch composition defects before the team runs, and what does it cost?*
Both benefits rest on the checker: RQ1's reliable "done" requires that the goal is covered by executed checks (W4, W5), and RQ2's lookup requires that the trace is well formed. RQ3 tests the checker on 151,516 mutants of builder-generated teams, compares it with LLM critics on independently seeded defects, and measures its run time. *Price measured:* sessions in which the builder never produces an admissible team, and milliseconds of checking.

Section **??** then collects the price that matters most to practitioners, task success, and reports a pre-registered experiment that tested whether a larger budget closes the success gap. It does not.

What we do not claim. We also planned to measure how often composition defects are the *decisive* cause of a failure and whether AutoM2M reduces such failures. Answering this requires labelling the cause of each failure. Our two LLM coders disagreed on the defect class in 65.6% of 5,580 sampled failures (Cohen's $\kappa = 0.16$), and we did not have the resources for human coding at this scale. We therefore make no claim about the causes of failures, and we drop the per-defect shares.

5.2 Implementation

This section describes how the concepts and algorithms of Section **??** are realised in the implementation that we evaluate, and where each of them can be found in the replication package. Table **??** maps every algorithm and component to its module.

Models and rules. AutoM2M and AgentHOT are written in Python. The compiler turns every view of an admitted team into a metamodel in PyEcore[1], an implementation of EMF's Ecore for Python, and every hand-off into a rule module in a small textual rule language with ATL's concrete syntax for matched rules, which the engine parses and interprets. Guards, structural bindings and footprints use an OCL fragment of navigation paths, literals, comparisons and conjunction. A footprint path that ends in an object, rather than an attribute, stands for all attributes of that object.

Builder. A typed team is exchanged as JSON, and the JSON Schema of this format plays the role of the typed-team metamodel: every proposal and every revision is decoded under it (Algorithm **??**, lines 4 and 12). The schema marks every property as required, because constrained decoding never emits an optional property and would otherwise never produce a guard. The builder's prompt library contains this schema, the validator library, three worked examples (Section **??**) and, for revisions, the navigation paths produced by TypedPaths, which follow at most two references. The anchor features of an obligation default to all attributes of its goal class.

Runtime. The prompt of a stochastic binding consists of the owner's role prompt, π_b, a hint on the expected output format, and the footprint serialised as *object.feature* = *value* lines. RunTeam stops after $p_{\max} = 4$ passes even if a model still changes; a binding that is still missing then counts as escalated. A footprint whose k_{samp} samples

[1] https://github.com/pyecore/pyecore

Table 5 Where each part of AutoM2M is implemented (paths relative to `src/` unless stated). The package also contains the experiment harness (`evaluation/run_matrix.py`), the studies of RQ2 (`evaluation/rq4/`) and RQ3 (`evaluation/rq2/`), whose directory names follow an earlier numbering, and the Slurm pipeline that ran them (`hpc/`).

Paper	Module	Main entry points
Algorithm **??**, AutoM2M loop	`autom2m/loop.py`	`AutoM2M.solve`, `_run_and_repair`, `_repair`
① Lifter, goal view MM_0	`autom2m/lift.py`	`lift_task`, `GOAL_VIEW`
Typed team (Def. **??**)	`autom2m/typed_team.py`, `autom2m/builder.py`	`parse_team`, `TEAM_SCHEMA`
Algorithm **??**, Build	`autom2m/loop.py`, `autom2m/builder.py`	`_build_v2`, `fragments_for`, `path_catalogue`, `apply_fixes`
Algorithm **??**, Admit	`autom2m/checker.py`	`Checker.w1`–`w6`, `check`
Validator library (Table **??**)	`autom2m/vlib.py`	`VLIB`, `v_test_valid`, `v_passes_tests`
⑤ Compiler	`agenthot/compiler.py`	`build_metamodels`, `generate_module`, `compile_team`
Algorithm **??**, runtime	`agenthot/team/runtime.py`, `agenthot/engine/executor.py`, `agenthot/engine/binding.py`	`run_to_fixpoint`, `run_handoff`, `apply_stochastic_binding`
Trace models, stamps (Def. **??**, **??**)	`agenthot/engine/trace.py`, `agenthot/rt_helpers.py`	`TraceLink`, `fpv`
Acceptance φ (Def. **??**)	`agenthot/session.py`	`Session._evaluate`, `_cover`, `_revalidate`
Algorithm **??**, Attribute	`autom2m/attribution.py`	`Attributor.locate`, `classify`, `attribute_all`
⑧ Repair, Apply	`autom2m/repair.py`	`plan`, `widen_delta`, `classify_delta`, `apply_delta`

were all rejected is remembered, so that the binding is not sampled again until its footprint changes. The deliverable $\delta = (D, g)$ names a target class and feature, here MethodImpl.body; the runtime assembles the deliverable by inserting every value of g into the class outline of M_0 (for function-level tasks, into the function signature). Behaviour validators run on this assembled artefact, so that a method is tested together with its siblings. Methods without an accepted value yet are replaced by stubs, and examples run in docstring order on one object, as doctest does, so that E2.1 can rely on the task that E1.1 added. A consequence is that a validator's verdict also depends on the sibling values, which are not part of the stamp; the clause *valid* therefore rechecks every value at the end of the run. When a method escalates, the deliverable still contains its last rejected candidate, so that every condition is scored on the code it produced; such a run never says done. Validators that execute code run in a separate process with resource limits and a time-out. The runtime does not restrict a validator to its owner's declared tools; tool sets are enforced only statically, by W6.

Validators. `test_valid` decides whether a test "asserts every example of m" by checking that the test contains each example's call and expected value as literals; this is a syntactic approximation, not a proof that the assertion is correct. Its other two parts, that the test runs and that it fails on a stub of m, are executed.

Attribution and repair. We use a replay cap of $n_{rep} = 3$ with the adaptive budget, a hop limit of $h = 2$, and three re-runs of the validator for the stability test. Per repair round, at most two failures are attributed, escalations first, then *valid* and then *cover*(G) failures. Replays in attribution are drawn without the rejection feedback of Algorithm **??**, line 21, so they sample from the footprint alone.

Versions. We distinguish two versions of the implementation. The *frozen* version produced every result of Sections **??**–**??**. It includes thirteen fixes made during a pilot with Qwen2.5-Coder 7B (`docs/experiment-log.md`) and the fixes made during the main runs (`hpc/logs/<run>/CHANGES.md`); both logs list, for every fix, the runs it invalidated, which were redone. The *revised* version adds the post-hoc changes R1–R3 of Section **??** and is used only in that section; the second log also records R1–R3 and the files they change. Both versions are tagged in the replication package.

5.3 Experimental design

We follow guidelines for experiments in software engineering [**?**] and for empirical studies involving LLMs [**?** **?**]: local open-weight models, fixed versions and decoding parameters, several seeds, all prompts and logs released, and cost reported alongside quality.

Subjects. *ClassEval* [**?**] contains 100 manually crafted Python class-level tasks with 412 methods and, on average, 33.1 hidden tests per class; a task succeeds only if *all* hidden tests of the class pass. Class-level tasks are where a team can plausibly help, because methods interact and must be designed, implemented and tested consistently. *HumanEval+* [**?**] contains the 164 HumanEval problems [**?**] with tests extended by EvalPlus. It represents small, self-contained tasks for which a team should add little. Teams see only the prompt, signatures, docstrings and the examples in the docstrings, which we call *public examples*; hidden tests are used only for scoring.

Models. We use seven instruction-tuned models from six families, all run locally through Ollama with 4-bit weights (tags in the package): Qwen2.5-Coder 7B and 32B [**?**], the agentic Devstral 24B [**?**], the general-purpose Gemma 3 27B [**?**], Granite-Code 20B [**?**], Code Llama 13B [**?**] and the completion-oriented StarCoder2 15B [**?**]. We chose them to vary the ability to emit a long, schema-conforming specification. We had planned to use DeepSeek-Coder-V2 16B [**?**]; under Ollama its generation stalled (requests timed out with a single output token), so we replaced it with Gemma 3 for this technical reason, before any of its results were available. In every configuration, the builder, all agents and the repair builder use the same model.

Conditions. Table **??** defines eight conditions. Single is the reference that every team must beat. Single-Gate is compute-matched to AutoM2M in its *budget*: it may spend up to AutoM2M's median output tokens for the same model and benchmark, and it stops at the first candidate that passes the public examples. Because it often stops early, it spends fewer tokens on average than AutoM2M (15.9k against 21.5k output tokens per ClassEval run). It also executes checks before declaring success, which lets us ask whether AutoM2M's reliability comes from executing checks alone. Free, Critic and Schema are re-implementations in the style of published multi-agent

Table 6 Experimental conditions. All conditions receive the same task text, the same worked examples rendered in their own format, and the same tools.

Condition	What runs	"Done" when
Single	one agent with code execution and two self-repair rounds	the agent stops
Single-Gate	Single, best-of-n under AutoM2M's token budget	public examples pass
Free	LLM-built prose team in a group chat	`TERMINATE`
Critic	Free, plus one revision after an LLM critic's review	`TERMINATE`
Schema	agents patch one shared JSON board (PatchBoard style)	`TERMINATE`
Typed-NC	builder's typed team, no build–check–repair loop	φ holds
AutoM2M	Algorithm **??**: build, check, run, attribute, repair	φ holds
Typed-Ref	hand-written typed team, checked and repaired	φ holds

systems with the same model, tools and examples, because those systems target other benchmarks and proprietary models; Critic's LLM reviews the team's JSON-rendered specification for D1–D5, and Free follows CaptainAgent. The two ablations isolate parts of AutoM2M. Typed-NC runs one schema-constrained proposal of the builder (re-prompted only if the JSON does not parse) with no checker, revision, attribution or repair; ill-typed bindings escalate, agent claims in φ are ignored, and the runtime stops after $p_{\max} = 3$ passes (4 for AutoM2M), because an unchecked team may contain cycles. It thus keeps the typed *format* and the engine-decided completion but removes the whole build–check–repair loop, so it measures the joint contribution of checking, ranking, revision and repair, not of the checker alone. Typed-Ref replaces the *builder* by a hand-written team; the same ClassEval team is used for HumanEval+, where each task has a single method.

Procedure. Every condition runs on every task of both benchmarks with every model and three seeds: $8 \times 7 \times 264 \times 3 = 44{,}352$ runs, on NVIDIA A100 GPUs of a national research cluster (Ollama version, model digests and job scripts are in the package). Temperatures are 0.2 for agents and bindings and 0.6 for the builder; outputs are capped at 4,096 tokens; the runtime draws up to $k_{samp} = 3$ samples per binding; the builder samples $k_{prop} = 3$ proposals and $k_{rev} = 2$ revisions per round for at most $k_{adm} = 3$ rounds; and AutoM2M and Typed-Ref make at most $k_{rep} = 2$ repair rounds, with two-sided W4. Builders receive three structurally different worked examples, with 2, 3 and 4 agents and none taken from either benchmark. In a pilot, one example's analyst asked for "edge cases and error conditions", and the builders' teams copied this into tests that demand exceptions the task never specifies. We reworded that example, in both its typed and its prose rendering, to ask only for documented behaviour, before the main runs.

Version of AutoM2M and a post-hoc revision. All results in Sections **??**–**??** use the version of AutoM2M that was frozen before the main runs. After seeing them, we revised AutoM2M to raise its success (three changes, described in Section **??**). Because the revision was designed with knowledge of the results, we piloted it only on a development subset (20% of each benchmark, drawn with a fixed seed when the revision began), evaluate it on the remaining 80% of the tasks in a pre-registered

experiment, and report it separately. These *held-out* tasks were not used to pilot the revision, but they were part of the main study whose aggregate results motivated it. The baselines were not changed.

Metrics. *Success*: all hidden tests pass. For RQ1 we treat each condition's own completion signal ("says done") as a classifier of success. A *run* is one condition on one task with one model and seed; for AutoM2M, it starts with one builder *session*, so sessions and runs correspond one to one. Every run then falls into one of four cells: it says done and passes (TP), says done and fails (FP), does not say done but passes (FN), or does not say done and fails (TN). With n runs, we report:

- *Silent-failure rate* FP/n: the share of *all* runs that say done but fail, that is, the rate at which a user who trusts the signal accepts a wrong result.
- *Flagged failures* $TN/(TN + FP)$: the share of failed runs that do not say done, that is, failures the condition admits.
- *Precision of "done"* $TP/(TP + FP)$: the share of runs that say done that pass.
- *Sensitivity of "done"* $TP/(TP + FN)$: the share of successful runs that say done. Its complement is the share of correct results that a user who trusts the signal would discard.
- *Matthews correlation coefficient* (MCC) between "says done" and success, which summarizes all four cells in one number that does not reward saying done rarely or often [**?**].

The first two measures improve when a condition says done less often; precision can too, if a condition says done only on easy tasks. We therefore read them together with sensitivity and MCC, and report how often each condition says done. *Cost*: input and output tokens and wall-clock time per run. Attribution and detection measures are defined in their sections.

Statistical analysis. Runs are clustered by task, because the same task appears in every condition, model and seed. All intervals on proportions are 95% percentile intervals from a task-clustered bootstrap with 1,000 resamples [**?**], unless stated otherwise; Wilson intervals [**?**] are used for small samples. Per model and benchmark, success is compared by exact McNemar tests [**?**] on task-level majority outcomes over the three seeds, Holm-corrected within each family of seven comparisons [**?**]. Paired comparisons of attribution and repair use exact McNemar tests. The pre-registered experiment of Section **??** uses logistic models fitted by generalised estimating equations (GEE), clustered by task [**?**]. We use $\alpha = 0.05$.

5.4 RQ1: Can the team's "done" be trusted?

Figure **??** shows, for every condition, how its runs divide into the four cells of the metrics; Table **??** in the appendix gives the silent-failure rate per model. We report five findings.

(1) AutoM2M has the fewest silent failures of all conditions. 5.9% [4.1, 7.8] of AutoM2M's ClassEval runs and 5.6% [4.3, 7.0] of its HumanEval+ runs say

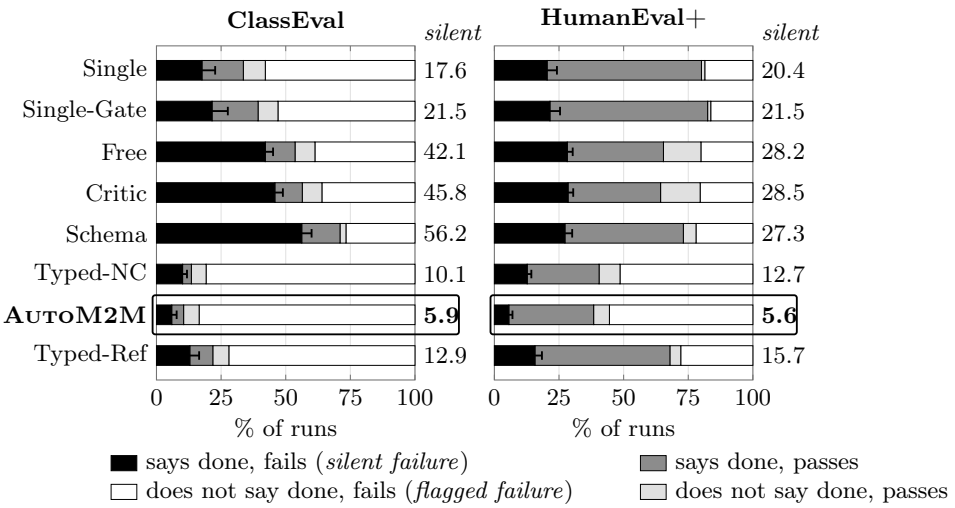


Figure 3 RQ1. Outcome of every run, split by the condition's own completion signal, pooled over seven models and three seeds (ClassEval: 2,100 runs per condition; HumanEval+: 3,444). Black: the run says done but fails the hidden tests (*silent failure*); its whisker is the 95% task-clustered bootstrap interval and its value is printed on the right. Dark grey: says done and passes. Light grey: does not say done but passes. White: does not say done and fails (*flagged failure*). The black and dark-grey segments together are the share of runs that say done.

done but fail. A single agent fails silently three to four times as often (17.6% [12.5, 22.5] and 20.4% [16.7, 24.2]), and free-form, critic and shared-schema teams five to ten times as often (27–56%). The pooled result holds per model: AutoM2M has fewer silent failures than Single for six of the seven models on both benchmarks; the seventh, StarCoder2, ties at zero because its single agent never says done. It has fewer than all three multi-agent baselines in 11 of the 14 model–benchmark pairs; the exceptions are Devstral's Free and Critic teams on both benchmarks, which rarely send `TERMINATE` at all, and Qwen2.5-Coder 7B's Schema team on HumanEval+.

(2) When AutoM2M fails, it says so. Of the failed AutoM2M runs, 93.4% (ClassEval) and 90.9% (HumanEval+) end without "done": the engine reports an unmet obligation, an open escalation or a refusal instead of a result. A failed single-agent run is flagged in 77% and 48% of cases, and a failed multi-agent run in 32–48%.

(3) Part of the advantage is abstention. AutoM2M says done in only 10.5% (ClassEval) and 38.4% (HumanEval+) of its runs, against 33.6% and 80.1% for Single, and a session whose team is never admitted (39% of AutoM2M's ClassEval sessions and 37% of its HumanEval+ sessions) cannot fail silently by construction. To separate this effect, we restrict AutoM2M to its admitted sessions (1,279 ClassEval and 2,178 HumanEval+ runs). Their silent-failure rate is 9.7% [6.9, 12.8] and 8.8% [6.9, 11.3]: higher than over all sessions, but still about half the single agent's rate. Per model, admitted teams fail silently less often than the single agent in 11 of the 14 model–benchmark pairs; StarCoder2 ties at zero on both benchmarks, and Qwen2.5-Coder 7B on HumanEval+ is the one exception (14.0% against 13.4%; Table **??**). The two builders that are admitted in almost every session show the same pattern with hardly

any refusals to explain it: their admitted teams fail silently in 15.1% (Qwen2.5-Coder 32B) and 17.4% (Devstral) of ClassEval runs, against 24.3% and 21.7% for their single agents, and in 6.6% and 11.9% of HumanEval+ runs, against 10.8% and 20.3%.

(4) As a signal of success, AutoM2M's "done" is the strongest on HumanEval+ and on par on ClassEval. On HumanEval+, 85.5% [81.6, 88.9] of AutoM2M's "done" runs pass, the highest precision of all conditions (single agent 74.5%, multi-agent teams 56–63%), and its MCC of 0.75 [0.72, 0.79] is well above that of every baseline (single agent 0.56 [0.50, 0.61]). On ClassEval, its precision (43.9% [31.2, 57.4]) and MCC (0.37 [0.26, 0.46]) are indistinguishable from the single agent's (47.7% and 0.38 [0.24, 0.51]) and about twice those of the multi-agent teams (precision 19–22%, MCC 0.02–0.15). The price of this caution is sensitivity: only 43.7% of AutoM2M's successful ClassEval runs say done (single agent: 65.4%), so a user who trusts the signal would discard more than half of its correct results. On HumanEval+, the corresponding shares are 84.3% and 97.8%.

(5) Executing tests before stopping is not enough. Single-Gate runs the public examples before it declares success, with the same budget cap as AutoM2M, yet it fails silently at least as often as Single (21.5% on both benchmarks). Passing the public examples is a weaker signal than φ, which requires every declared obligation to have been checked and every accepted value to be re-validated on the final deliverable. The ablation Typed-NC, which keeps the typed format and φ but removes the build–check–repair loop, fails silently more often than AutoM2M when pooled (10.1% against 5.9% on ClassEval; 12.7% against 5.6% on HumanEval+) and has lower precision (25.0% against 43.9%; 68.7% against 85.5%). Per model, however, the picture is mixed (Table **??**). Typed-NC has *fewer* silent failures than AutoM2M in 6 of the 14 model–benchmark pairs, including both strong builders on both benchmarks (for example 6.3% against 14.7% for Qwen2.5-Coder 32B on ClassEval), and as many or more in the others. The pooled difference comes mainly from Granite-Code, Code Llama and StarCoder2, whose unchecked teams fail silently in 7–36% of runs while their AutoM2M sessions are mostly refused or never reach φ. We therefore read the ablation as evidence that the typed format and engine-decided completion carry much of the reliability by themselves, and that the build–check–repair loop mainly protects against weak builders; the ablation does not isolate the checker. For the same reason, the hand-written Typed-Ref team, which is checked but used for every task, can fail silently more often (12.9% on ClassEval) than the unchecked Typed-NC teams, which rarely reach φ at all (13.5% say done, against 21.8% for Typed-Ref).

The price. AutoM2M's reliability comes with a more conservative "done" and with lower success. It succeeds in 10.6% (ClassEval) and 38.9% (HumanEval+) of runs, against 24.5% and 61.0% for a single agent (Section **??** explains why). It spends 7 times (ClassEval) and 16 times (HumanEval+) the output tokens of a single agent (21.5k against 2.9k and 12.7k against 0.8k per run on average), and 14 times its input tokens on ClassEval (65.8k against 4.6k), because every binding is prompted separately with its footprint.

Answer to RQ1. Largely yes. Of the eight conditions, AutoM2M has the fewest silent failures on both benchmarks (5.9% and 5.6% of runs, a third to a quarter of a single agent's rate) and flags more than 90% of its failures. Part of this comes from abstention, but admitted teams still fail silently about half as often as a single agent (9.7% and 8.8%). The typed format alone, without the build–check–repair loop, already removes much of the gap. Its "done" is the most reliable signal of success on HumanEval+ and as reliable as a single agent's on ClassEval. *Price:* AutoM2M says done less often, discards more than half of its correct ClassEval results, succeeds less often, and uses 7–16 times the output tokens of a single agent.

5.5 RQ2: Can failures be traced, explained and repaired?

Setup. To know the true cause of a failure, we injected it. The study has three parts.

- *Fault injection.* We took runs that had succeeded and injected one fault of one of five classes: Upstream, a wrong upstream value accepted by a weak validator; Footprint, the deliverable's footprint narrowed to the method signature; Specification, an unsatisfiable validator; Sampling, the owner's LLM degraded for one method (each answer is replaced by a broken one with probability 0.6, also during replays, giving pass rates of about 0.3–0.6); and Validator, a flaky validator. Each fault is injected into a snapshot of the clean run, and only what the injection makes stale is re-sampled, so the faulted run differs from the clean run only by the injection. A fault counts if φ then fails at a value that the clean run had accepted and still held valid at its end (for Upstream, at the value downstream of the corrupted one). We injected faults into ClassEval runs of the hand-written Typed-Ref team with Qwen2.5-Coder 7B and 32B and Devstral, and into runs of builder-generated AutoM2M teams with Qwen2.5-Coder 32B. Of 1,108 injections, 993 produced a failure (752 in reference teams, 241 in builder teams; 47–153 per class).
- *Transcript-based baselines.* Four methods from Who&When and TraceElephant [**?** **?**] (all at once, step by step, binary search and agentic replay) received the same 752 faulted reference-team runs as transcripts with full inputs: one step per LLM call, with its agent, input, output and the automatic check of the output. All four used Qwen2.5-Coder 32B as judge and the five classes, with one-line definitions, as a closed label set.
- *Repair.* We took 562 failed AutoM2M runs and compared one checked repair of the diagnosed part (Section **??**) with rebuilding the whole team from scratch given the same fault report.

(1) The trace locates the responsible part. Table **??** reports the results; the first three transcript methods come from Who&When [**?**] and agentic replay from TraceElephant [**?**]. With 752 failures, every percentage in the table has a Wilson 95% interval of at most ±3.6 points (±5.3 for the 241 builder-team failures), so the differences between trace and transcript methods are far larger than the noise. The binding where φ fails (the symptom) is found in every case, by construction (Proposition **??**). The trace finds the binding where the fault was *injected* in 98.3% of

Table 7 RQ2. Attribution of 993 injected faults (%). *Responsible binding*: the injected binding is found (transcript methods: the responsible step). *Responsible agent*: the responsible agent is named. *Fault class*: macro accuracy over the five fault classes (chance: 20%). *LLM calls*: calls per failure.

Method	Responsible binding	Responsible agent	Fault class	LLM calls
*Trace-based (*AutoM2M*), reference teams,* $n = 752$				
adaptive replay (≤ 3)	**98.3**	**77.9**	**75.3**	7.7
single replay	98.3	77.9	65.7	3.7
*Trace-based (*AutoM2M*), builder teams,* $n = 241$				
adaptive replay (≤ 3)	96.3	77.6	73.7	7.8
Transcript-based, reference teams, $n = 752$				
all at once	60.2	55.1	32.0	1.0
step by step	53.6	41.5	13.0	12.9
binary search	61.0	45.7	32.8	5.1
agentic replay	61.8	52.3	22.9	3.0

reference-team failures and 96.3% of builder-team failures, including 92.5% and 83.3% of upstream faults, whose symptom appears one hand-off later. The best transcript method locates the responsible step in 61.8% of cases. Results are stable across the three models of the reference team (responsible binding 96–100%). The responsible *agent* is named in 78% of cases, against at most 55% by transcript methods. The gap between the binding (98%) and the agent (78%) has one cause: every binding has a single owner (W2), but when attribution blames the *validator*, it names no agent. Of the 153 reference-team failures with the correct binding but the wrong agent, 152 are sampling faults taken for validator faults or the reverse.

(2) The trace explains what kind of fault occurred. With adaptive replay, AutoM2M names the fault class correctly in 75.3% of reference-team failures and 73.7% of builder-team failures, more than twice the best transcript method (32.8%). Chance is 20%; the step-by-step method (13.0%) stays below it because in 359 of the 752 runs (48%) it judges no step decisive and therefore returns no class. Specification, upstream and footprint faults are classified correctly in 83–100% of cases. The hard case is telling Sampling faults (an agent that fails by chance) apart from Validator faults (a check that fails by chance): both can make a replay pass, so in reference teams 43% of sampling faults are taken for validator faults and 57% of validator faults for sampling faults. The replay budget matters for exactly this case. With a single replay, sampling faults are recognised in 7% of cases, because the injected agents fail a single replay 40–70% of the time: in 74 of the 152 cases, the one replay on the original footprint fails and the one replay on the widened footprint passes, so the fault is taken for a footprint fault. With up to three replays, sampling faults are recognised in 57% of cases. Macro accuracy rises from 65.7% to 75.3% (77 against 4 discordant failures, exact McNemar $p < .001$).

(3) Repairing the diagnosed part is as successful as rebuilding, with less work. One checked repair made the task pass its hidden tests in 21.5% [18.3, 25.1] of the 562 failed runs, against 18.9% [15.8, 22.3] for rebuilding the team from scratch

(difference 2.7 points [−0.4, 5.7]; 49 against 34 discordant runs, exact McNemar $p = .12$). The interval excludes a disadvantage of more than half a point, so repair is not worse than rebuilding for any practically relevant margin. The repair needed a median of 2,309 tokens instead of 4,167 (45% fewer), and it preserved 79% of the values that the team had already accepted. Of the repairs, 72% were applied in place, that is, only the diagnosed rule or binding changed. Repair was never worse than rebuilding for any model (Qwen2.5-Coder 7B: 16.9% against 10.6%; Devstral: 20.0% against 18.4%; Qwen2.5-Coder 32B: 27.9% for both).

A caveat on the comparison. The five injected classes are the five classes that Algorithm **??** distinguishes, and three of the injections (flaky validator, unsatisfiable validator, narrowed footprint) correspond directly to one of its tests. The comparison therefore shows that the trace and bounded replay recover a known taxonomy of faults far better than a transcript does; it does not show how well this taxonomy covers natural failures. The transcript methods were also adapted to our runs and to a 32B judge rather than used on their native conversations with the proprietary judges of their original papers.

The price. Traceability is not free. Adaptive attribution uses 7.7 LLM calls per failure, against one call for the cheapest transcript method; for Qwen2.5-Coder 32B, attribution takes a median of 793 s in the runs that invoke it, which is more than half of the median run (1,444 s), because replays re-run stochastic bindings. Repair also helps only a minority of failed runs: four in five failures remain failures after one repair round.

Answer to RQ2. Yes, for the fault classes we injected. The trace locates the responsible binding of 96–98% of injected faults, against at most 62% (step) for transcript methods, and names the fault class correctly in 74–75% of cases, against at most 33%. A checked repair of the diagnosed part is as successful as rebuilding the team, uses 45% fewer tokens and keeps 79% of the accepted work. *Price:* about eight LLM calls per failure; with Qwen2.5-Coder 32B, attribution takes more than half of the median run time, and four in five failures remain failures after one repair.

5.6 RQ3: Are composition defects caught before the team runs?

Setup. We test the checker in three ways.

- *Independently seeded defects.* To avoid testing the checker only on defects designed for it, Gemma 3 was asked to seed one defect into each of 300 admitted builder teams (60 per defect class). It saw only the plain-language definitions of D1–D5 (Table **??**), never W1–W6. When its edit left the team unchanged, it was told so and asked again, up to four times; 23 teams that were still unchanged and 2 for which no team came back were removed, leaving 275 seeded teams (48–59 per class). To measure false alarms, we built a clean set *without* the checker: 30 hand-written typed teams for ClassEval classes and 30 teams of the unchecked condition Typed-NC that reached φ and passed the hidden tests. Three LLM critics (Qwen2.5-Coder 7B and 32B, Devstral) received the same JSON specifications and defect definitions as the seeder.

- *Mutation of natural teams.* We wrote nine mutation operators, one or two per defect class, and applied each at every applicable site of the 3,379 distinct teams that builders generated and the checker admitted in the main runs (3,457 admitted sessions, some of which produced identical teams), yielding 151,516 mutants (Table **??**). Each mutant was checked with three variants of W4: *path-only* (some class-level path leads from each goal class to a behaviour-checked class), *one-sided* anchoring (goal data must reach the check of a value), and the *two-sided* anchoring of Definition **??** (goal data must reach both the producer and the check).
- *Mutation of hand-written teams.* The same operators applied to the typed DevTeam and a hand-written requirements team that derives user stories and API operations from a goal model (85 mutants), under two goal declarations: G_1 declares only checked obligations, and G_2 also declares the deliverables.

(1) On independently seeded defects, the checker is the only judge that separates defective from clean teams. With two-sided anchoring, the checker rejects 173 of the 275 seeded teams (62.9%, Wilson 95% CI [57.1, 68.4]) and raises false alarms on 8 of the 60 clean teams (13.3% [6.9, 24.2]; one-sided and path-only: 11.7%). Detection depends on the defect class: 98% for unverifiable completion (D4), 71% for coverage gaps (D1), 66% for ownership conflicts (D3), 47% for capability mismatches (D5) and 27% for hand-off mismatches (D2). The three variants of W4 perform alike here, because no seeded defect detaches the goal from the producers. The LLM critics do not discriminate at all. The two larger critics report a defect in *every* team, seeded or clean, and the 7B critic reports one in 40.4% of seeded and 40.0% of clean teams. A critic that flags every team, or flags teams at random, gives an engineer no information about a specific team.

What the checker misses, and its false alarms. Reading the descriptions of the 102 seeded defects that the checker misses, we grouped them roughly as follows. About 40% add an agent or a tool that no hand-off uses, which the seeder labelled as an ownership conflict (D3) or a capability mismatch (D5). Such an addition does not break the team, so arguably it is not a composition defect at all; without these cases, detection would be about 74%. About 20% change a prompt's wording or a declared output format but no footprint, for example asking for a return type "as a string"; the checker reasons about what an agent reads and writes, not about what its prompt says. About 15% remove a guard, which the checker only warns about (Section **??**). The rest change behaviour that no static check can see, for example an agent instructed to introduce off-by-one errors. All eight false alarms are Typed-NC teams, which were written without the checker and happened to pass their task. Four reference a feature that their views do not declare (W1), three let two agents write one view (W2), and three have a producer that does not read the goal (W4); some violate two rules. These teams passed their task despite the violations. The checker thus rejects some teams that would have worked; this is the price of a conservative check.

(2) Mutation analysis: a sanity check and a blind spot of weaker coverage. High detection of mutants is expected by construction, because each operator breaks a rule that the checker states. The analysis is nonetheless informative in two ways. First,

Table 8 RQ3. Rejection rate (%) of 151,516 mutants of 3,379 builder-generated admitted teams, under three variants of W4 (all other rules unchanged). Defects as in Table **??**.

			W4 variant		
Defect	Mutation operator	Mutants	Path-only	One-sided	Two-sided
D1	delete a rule	10,387	99.8	99.9	99.9
D1	detach goal from footprints	3,377	**0.0**	**1.4**	**99.97**
D2	break a footprint path	42,677	100	100	100
D2	drop a mandatory binding	30,509	100	100	100
D3	add a second writer	23,533	100	100	100
D3	duplicate a producing rule	10,387	100	100	100
D4	add an agent claim to φ	10,405	100	100	100
D4	drop an engine clause	13,508	100	100	100
D5	remove a required tool	6,733	99.8	99.8	99.8
	all operators	151,516	97.75	97.79	**99.99**

it confirms that the rules behave as specified on teams written by LLMs, not only on our own: with two-sided anchoring, the checker rejects 151,496 of the 151,516 mutants (99.99%); the 20 survivors are 12 tool removals, 7 rule deletions and one detached team. Second, it exposes a blind spot of weaker formulations of coverage (Table **??**, second row). The *detach* operator keeps every rule but removes all goal data from the producers' footprints, so agents write code without ever seeing the task's examples. Path-only coverage admits all 3,377 detached teams, and one-sided anchoring admits 98.6% of them, because the validators still read the examples. Only two-sided anchoring rejects them. The hand-written teams add a second lesson: under G_1, deleting the rule that creates the requirements team's API operations escapes all variants, because every declared obligation still reaches a check; declaring the deliverables (G_2) catches it, and two-sided anchoring then rejects all 85 mutants. Coverage is relative to what is declared.

(3) Checking costs milliseconds. Figure **??**a shows that the cost of a check grows polynomially with team size, as Proposition **??** predicts: chain teams of 1,000 and 3,000 views are checked in 49 ms and 262 ms, DAGs with fan-in three in 105 ms and 551 ms (local growth exponents 1.53 and 1.50). At 1,000 views, most of the time goes to typing (W1: 20 ms of 49 ms for chains, 53 ms of 105 ms for DAGs) and to anchoring (W4: 17 ms and 34 ms); W2, W3, W5 and W6 together take less than 20 ms. Teams that builders actually generate have three to six views. An AutoM2M run performs a median of six checks (proposals, revisions and repair deltas), which take a median of 3.0 ms in total (95th percentile 7.3 ms). Figure **??**b puts this into perspective: in a median Qwen2.5-Coder 32B ClassEval run of 1,444 s, checking accounts for a share of 2×10^{-6}, whereas the LLM-written bindings (472 s) and, when it is invoked, attribution (793 s) dominate. The cost of AutoM2M is the cost of its LLM calls, not of its checks.

The price: weak builders often produce no team. A checker that rejects defective teams also rejects the only teams that a weak builder can write. Figure **??** shows a sharp divide. Qwen2.5-Coder 32B and Devstral produce an admissible team in 96–97%

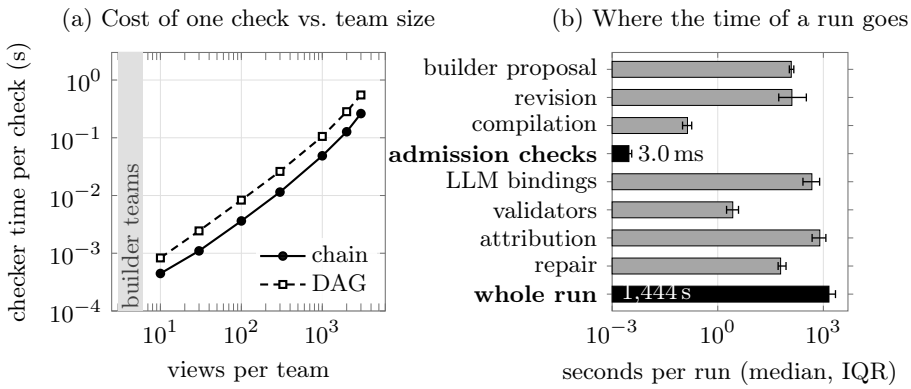


Figure 4 RQ3, cost of checking. (a) Median checker time per check on synthetic teams of growing size, 10 runs per size, log–log: *chain* teams, in which view i reads view $i-1$, and *DAG* teams, in which each view reads three earlier views. The grey band marks the size of the teams that builders actually generate (3–6 views). (b) Wall-clock time of an AutoM2M run by step, Qwen2.5-Coder 32B on ClassEval (292 admitted runs): median (bar) and interquartile range (whisker), log axis; revision, attribution and repair over the runs that invoke them. Black bars: the checks of a whole run, and the whole run.

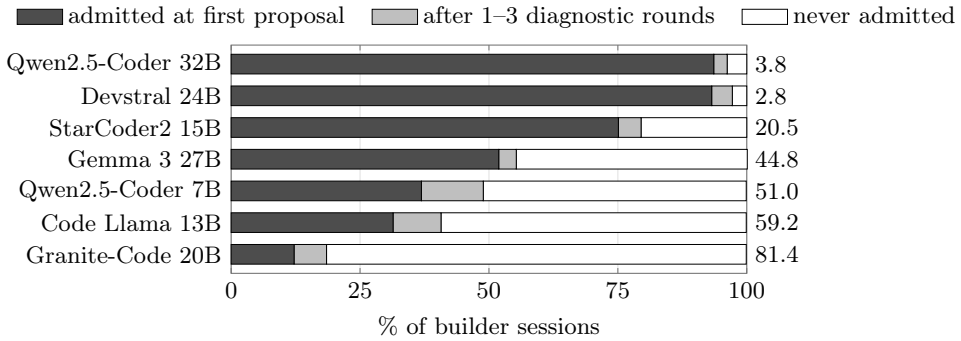


Figure 5 RQ3, the price of prevention: outcome of the builder sessions per model, both benchmarks pooled (792 sessions per model), sorted by the share of admitted sessions. Right: share of sessions never admitted within $k_{adm} = 3$ revision rounds (%).

of sessions, nearly always at the first proposal, and StarCoder2 in 80%. Gemma 3, Qwen2.5-Coder 7B, Code Llama and Granite-Code reach only 19–55%. Across the 8,851 rejected proposals, the first violation was W1 (a reference to a feature that the builder never declared) in 73%, W4 in 14% and W2 in 12%. These are mostly inconsistencies within a single proposal, not poor design decisions. Diagnostic rounds help little: they add 3–12 points of admission. A refused session is an honest outcome: the team does not run and does not claim success. It is also lost work, and one of the two reasons for AutoM2M's lower success (Section **??**).

Table 9 Task success (%) on ClassEval (top) and HumanEval+ (bottom), mean over 3 seeds. Bold: best condition per model. A mark on a cell compares AutoM2M with the condition of that cell: $^{\downarrow}$ AutoM2M is significantly lower, $^{\uparrow}$ significantly higher (exact McNemar on task-level majority outcomes, Holm-corrected over seven comparisons per model).

Model	Single	Single-Gate	Free	Critic	Schema	Typed-NC	AutoM2M	Typed-Ref
CodeLlama	15.7$^{\downarrow}$	**17.3**$^{\downarrow}$	10.0$^{\downarrow}$	10.7$^{\downarrow}$	0.0	0.3	1.3	7.0$^{\downarrow}$
Devstral	33.0	**35.0**	29.3	28.3	26.0	17.3	23.3	22.3
Gemma3	**34.3**$^{\downarrow}$	32.3$^{\downarrow}$	29.0$^{\downarrow}$	25.7$^{\downarrow}$	24.3$^{\downarrow}$	11.7	11.0	20.7
Granite	20.0$^{\downarrow}$	**20.3**$^{\downarrow}$	15.7$^{\downarrow}$	14.7$^{\downarrow}$	5.7	0.7	4.3	13.0$^{\downarrow}$
Qwen-32B	36.0	**38.7**$^{\downarrow}$	27.0	28.0	30.7	25.3	25.3	28.7
Qwen-7B	32.7$^{\downarrow}$	**34.7**$^{\downarrow}$	21.0$^{\downarrow}$	18.7$^{\downarrow}$	23.0$^{\downarrow}$	8.3	8.7	14.0
StarCoder2	0.0	0.0	2.7	1.3	**10.3**$^{\downarrow}$	0.0	0.0	0.0
Mean	24.5	25.5	19.2	18.2	17.1	9.1	10.6	15.1

Model	Single	Single-Gate	Free	Critic	Schema	Typed-NC	AutoM2M	Typed-Ref
CodeLlama	40.4$^{\downarrow}$	**42.5**$^{\downarrow}$	23.4	22.6	10.6$^{\uparrow}$	23.4	19.7	42.1$^{\downarrow}$
Devstral	77.2	**79.3**	77.2	76.0	75.0	71.1	77.0	71.5$^{\uparrow}$
Gemma3	81.3$^{\downarrow}$	**81.9**$^{\downarrow}$	72.0$^{\downarrow}$	71.3$^{\downarrow}$	77.4$^{\downarrow}$	50.2	49.0	69.5$^{\downarrow}$
Granite	57.5$^{\downarrow}$	**60.2**$^{\downarrow}$	37.4$^{\downarrow}$	37.0$^{\downarrow}$	27.4$^{\downarrow}$	4.7	10.6	56.9$^{\downarrow}$
Qwen-32B	**88.0**	87.4	80.1	79.7	83.7	76.0	83.3	86.2
Qwen-7B	82.7$^{\downarrow}$	**83.9**$^{\downarrow}$	68.9$^{\downarrow}$	67.9$^{\downarrow}$	62.8$^{\downarrow}$	24.0	30.3	66.1$^{\downarrow}$
StarCoder2	0.0	0.2	2.6	3.3	**18.1**$^{\downarrow}$	1.6	2.4	2.4
Mean	61.0	62.2	51.7	51.1	50.7	35.9	38.9	56.4

Answer to RQ3. Yes, for the defects it is designed for. On independently seeded defects, the checker catches 63% with 13% false alarms and is the only judge whose verdict separates defective from clean teams; the LLM critics flag clean and defective teams alike. On 151,516 mutants of LLM-written teams it behaves as specified (99.99% rejected), and only two-sided anchoring catches teams whose producers never see the goal. A check takes milliseconds. *Price:* for four of seven models, 45–81% of builder sessions never produce an admissible team.

5.7 The price in task success, and whether a larger budget pays it

Success in the main study. Table **??** reports success per model. AutoM2M succeeds less often than the single-agent baselines, with mean success of 10.6% (ClassEval) and 38.9% (HumanEval+) against 24.5% and 61.0% for Single. Two causes contribute, and their weight depends on the builder. The first is *refusal*: every session whose team is never admitted is a failed task. It dominates for the weak builders: Granite-Code is admitted in 14% of its ClassEval sessions and Code Llama in 29%, so even their admitted teams (30% and 5% success) cannot raise the total. The second is the *decomposition* itself: admitted teams succeed less often than a single agent on ClassEval even for strong builders (Qwen2.5-Coder 32B: 26.0% against 36.0%; Devstral: 23.9% against 33.0%), and so does the hand-written Typed-Ref team, which involves no builder

(15.1% against 24.5% on average). A typed team implements each method separately from its footprint, and whatever the footprint omits, for example how sibling methods share state, a single agent that sees the whole class can still use. On HumanEval+, where a task is a single function, admitted teams of strong builders are on par with a single agent (Qwen2.5-Coder 32B: 87.2% against 88.0%; Devstral: 79.5% against 77.2%), and for these two models AutoM2M as a whole is on par with the multi-agent baselines (Qwen2.5-Coder 32B: 83.3% against 79.7–83.7%; Devstral: 77.0% against 75.0–77.2%; no significant difference). A compute-matched single agent with an execution gate (Single-Gate) has the highest mean success on both benchmarks, in line with reports that the gains of multi-agent systems over strong single agents are small [**? ?**].

A post-hoc revision. After seeing these results, we revised AutoM2M in three ways that target the lost work rather than the checker:

(R1) after admission, the runtime extends the footprint of the deliverable binding by the goal method's docstring and examples, if the builder left them out, and adds an `examples_run` validator; this edit only adds reads and a check, but it is applied to the admitted team without a second check;
(R2) when a method escalates, the deliverable contains the rejected candidate that passes the most documented examples, instead of the last rejected candidate;
(R3) when the builder's team is not admitted, AutoM2M runs a hand-written, admitted library team (the Typed-Ref team) instead of refusing.

The checker, the anchoring criterion and the baselines are unchanged. On the held-out tasks, the revision raised AutoM2M's success with Qwen2.5-Coder 7B from 9.6% to 21.2% (ClassEval) and from 28.8% to 71.8% (HumanEval+), and with Qwen2.5-Coder 32B from 23.8% to 30.0% and from 81.9% to 86.8%. The frozen values are the main-study runs on the held-out tasks ($k_{samp} = 3$); the revised values are those of Experiment 1 at $k_{samp} = 1$ (Table **??**), so the gain is not due to more samples. The comparison is exploratory.

The revision also shows the trade-off between success and reliability (same runs). With Qwen2.5-Coder 32B, it kept the reliability advantage: silent failures of 7.9% (ClassEval) and 5.9% (HumanEval+) against 28.8% and 10.4% for Single-Gate. With Qwen2.5-Coder 7B on HumanEval+, it did not: falling back to the library team and keeping best-effort candidates raised silent failures from 5.9% (frozen version) to 16.3%, above Single-Gate's 13.5%. Recovering lost work can thus reintroduce the silent failures that the checker prevents, when the builder is weak.

Experiment 1: does a larger budget close the gap?. A typed team validates and repairs each method separately, whereas a single agent must get a whole class right in one sample. We therefore expected AutoM2M's success to rise with its budget faster than Single-Gate's, especially for large classes. We pre-registered this experiment: hypotheses, design and analysis were written down on 22 October 2026, before any of its runs, and the protocol is in the package [registry link to be added]. The hypotheses were: *H1*: on ClassEval, the success of AutoM2M rises more steeply with $\log_2 k$ than that of token-matched Single-Gate; *H2*: the difference is larger for classes with at least

Table 10 Pre-registered Experiment 1 on held-out tasks (ClassEval: 80 tasks, HumanEval+: 131 tasks; 3 seeds): success (%) of the revised AutoM2M and of Single-Gate with k samples per binding; Single-Gate's budget is AutoM2M's median output tokens at the same k.

		AutoM2M				Single-Gate			
Benchmark	Model	k=1	2	4	8	k=1	2	4	8
ClassEval	Qwen 7B	21.2	18.8	18.8	17.5	35.8	37.5	37.5	35.0
	Qwen 32B	30.0	29.2	30.4	31.2	40.4	40.8	40.8	41.2
HumanEval+	Qwen 7B	71.8	70.7	74.0	72.8	84.5	84.5	83.7	84.5
	Qwen 32B	86.8	88.5	89.1	87.0	88.5	89.1	88.5	89.1

six methods than for classes with at most three; *H3* (negative control): on HumanEval+ there is no such difference. We varied the samples per binding $k \in \{1, 2, 4, 8\}$, gave Single-Gate AutoM2M's median output tokens at each level as its budget cap, used the revised AutoM2M, and ran on the held-out tasks with three seeds. Of the 80 held-out ClassEval classes, 30 have at most three methods and 13 at least six. For each benchmark and model, we fitted a logistic GEE with an exchangeable working correlation, clustered by task, with the terms condition, $\log_2 k$ and their interaction (for H2, also class size and all its interactions); the interaction coefficient is on the log-odds scale per doubling of k.

The hypotheses are not supported (Table **??**; for reference, a single agent reaches 35.8% and 37.1% on ClassEval and 84.0% and 88.0% on HumanEval+ with the 7B and 32B model on the same tasks, and a free-form team 21.7%, 27.9%, 69.0% and 78.9%). For Qwen2.5-Coder 7B, AutoM2M's success on ClassEval does not rise with the budget; it falls slightly from 21.2% to 17.5% while its output tokens grow from 21.9k to 30.9k. The condition $\times \log_2 k$ interaction is -0.06 (95% CI $[-0.17, 0.04]$, odds ratio 0.94 per doubling, $p = .24$; H1 not supported); the interval rules out a gain of more than 0.04 on the log-odds scale per doubling, and the three-way interaction with class size is 0.09 ($p = .40$; H2 not supported). Both conditions plateau: the change from $k = 4$ to $k = 8$ is -1.3 points $[-5.4, 2.5]$ for AutoM2M and -2.5 points $[-5.8, 0.4]$ for Single-Gate. Qwen2.5-Coder 32B shows the same flat pattern on ClassEval: AutoM2M stays between 29.2% and 31.2% while its output tokens grow from 17.5k to 24.7k, and Single-Gate stays between 40.4% and 41.2%. The interaction is 0.01 ($[-0.08, 0.10]$, $p = .77$; H1 not supported), and the three-way interaction with class size is 0.17 ($p = .24$; H2 not supported). On HumanEval+, as expected under H3, no slope difference appears (Qwen2.5-Coder 7B: 0.04, $p = .37$; Qwen2.5-Coder 32B: 0.00, $p = .97$). With Qwen2.5-Coder 32B on HumanEval+, the revised AutoM2M stays within about two points of Single-Gate at every level and exceeds it at $k = 4$ (89.1% against 88.5%). More samples per binding do not help when the samples fail for the same reason, for example because the agent misreads the docstring in every sample.

The price in success. AutoM2M is not a way to raise task success. In the main study, it succeeds less often than a single agent, because weak builders often cannot write admissible teams and because, on class-level tasks, even admitted and hand-written typed teams succeed less often than a single agent; on function-level tasks, strong builders are on par with multi-agent teams. A post-hoc revision recovers much of the lost work on held-out tasks but, with a weak model, also part of the silent failures. A larger budget does not close the gap: the pre-registered hypotheses were not supported for either model.

6 Discussion

What AutoM2M offers: a team whose results can be trusted and explained. RQ1 and RQ2 support the two benefits, and RQ3 the mechanism behind them: the typed format makes completion a property that the engine computes, and the checker rejects teams whose goal is not covered by executed checks. The Typed-NC ablation suggests that both parts contribute, and that the effect of the checking loop differs between builders. Both benefits matter when the cost of an undetected wrong result is high, for example when the output of a team flows into a larger system without review, when several teams run unattended, or when a failure must be explained to a person rather than merely retried.

Why executing tests is not enough. One could expect any condition that runs tests before declaring success to be as reliable as AutoM2M. Single-Gate refutes this: it executes the public examples under the same budget cap, yet it fails silently more than twice as often as admitted AutoM2M teams. The difference is coverage. Single-Gate stops when the examples it happens to run pass; AutoM2M's φ is true only when every declared obligation has been checked, and two-sided anchoring ensures that the checks and the producers both see the goal. The detach mutants of RQ3 show why the second side matters: without it, a team can pass its checks while its producers never saw the task's examples.

What AutoM2M does not offer. AutoM2M does not make agents reason better, and it does not raise task success over a single agent; it trades success for reliability. The trade-off is clearest for weak builders, whose sessions are refused, and in the post-hoc revision, where recovering refused sessions reintroduced silent failures for the weakest model. Its caution also has a cost of its own: more than half of its correct ClassEval results are not reported as done. We found no support for two further benefits that we had hoped for: more budget did not help (Experiment 1), and we cannot say whether AutoM2M reduces composition-caused failures, because LLM coders did not agree on failure causes (Section **??**).

When to use AutoM2M. The results suggest a simple rule. If the goal is the highest success rate per token, a single agent with an execution gate is the better choice. If the goal is a result that can be trusted without re-checking, and failures that can be located and repaired, AutoM2M with a capable builder offers both, at 7–16 times the output tokens of a single agent. With a weak builder, AutoM2M remains honest, since refused sessions do not claim success, but it does little useful work.

Builders and decomposition are the bottlenecks. Admission divides the models sharply, and the most frequent first violation is a reference to a feature that the builder never declared, an inconsistency within one proposal. Because the checker's diagnostics are precise and cheap, they could serve as a training signal for builders, which we have not explored. Builders also imitate the worked examples (Section **??**), so the examples shape the teams as much as the task does. Even admitted and hand-written teams lose success on class-level tasks, which suggests that per-method footprints omit context that a single agent uses; footprints that include the class's shared state are a direct design option.

Implications. *Researchers* evaluating multi-agent systems should report silent failures and the precision of "done" alongside success, and should include a compute-matched single agent with an execution gate as a baseline. *Framework builders* could run W2, W4 and W5 as a lint over LangGraph or CrewAI teams, whose agents, typed state and edges map onto views, hand-offs and write rights; we have not evaluated such a lint. *Practitioners* who need trustworthy results should prefer completion signals that are computed from executed checks covering the goal over signals that agents emit.

Future work. The revised AutoM2M was evaluated on two models only. Running it on all seven models and both benchmarks, and finding a recovery strategy for refused sessions that does not reintroduce silent failures, is the most direct next step. Others are builders trained on the checker's diagnostics, validators whose adequacy is itself checked, and a human-coded study of failure causes.

7 Threats to Validity

Construct validity. Each condition ends a run with its own signal, so the reliability measures compare different mechanisms. This is intended: a user sees exactly this signal. Silent and flagged failures improve when a condition says done rarely, and refused sessions can never fail silently; we therefore report precision, sensitivity and MCC with them, and repeat the analysis on admitted sessions only. Hidden tests are an imperfect oracle [**?**], and a run that passes them may still be wrong. Injected faults (RQ2) and mutants (RQ3) are synthetic, and the injected fault classes coincide with the classes that attribution distinguishes; we mitigated this by injecting into real runs, by mutating teams that LLMs wrote, and by having a different model seed defects from plain-language definitions, but the attribution of natural failures remains untested. The seeded defects are labelled by the seeder, and some of them are arguably not defects (Section **??**). `test_valid` recognises the examples in a test syntactically, so a test that names an example without asserting it correctly can be accepted.

Internal validity. The baselines are re-implementations of published systems, and the runtime, checker, baselines and reference team come from the same author. Typed conditions run on AgentHOT and the others in a group-chat harness with the same time-outs and sandbox. The Typed-NC ablation removes the whole build–check–repair loop, not only the checker. Re-samples after a rejection see the rejected value and the validator's message, whereas replays in attribution do not, so a replay may succeed less often than the original sampling and push a fault towards Specification. The

runtime does not confine validators to their owner's declared tools, so W6 is a property of the specification only. The revision of Section **??** was designed after seeing the main results; we piloted it on a development subset, evaluated it on the other tasks, did not change the baselines, and report it separately. Fixes were made during the pilot and the main runs; each invalidated run was redone, and both logs are in the package. One model (Gemma 3) replaced a planned model for a technical reason before any of its results existed. LLM outputs are non-deterministic [**?**]; we use three seeds and task-clustered intervals.

External validity. Results may depend on the three worked examples given to builders, whose choice we did not vary: 71% of the admitted ClassEval teams (911 of 1,279) reuse the agents of one of the examples, although 207 distinct team shapes appear. We study only Python tasks with docstring examples, which favour executable validators and which the lifter relies on, and open-weight models of up to 32B parameters, run with 4-bit quantisation. Larger or unquantised models may write admissible teams more often, which would reduce the price of AutoM2M but not necessarily change its reliability. The transcript-based attribution methods were run with a 32B judge, not with the proprietary models of their original papers.

Conclusion validity. We report task-clustered intervals and exact paired tests, and Holm-correct the pairwise success comparisons. The success comparisons use task-level majority outcomes over the three seeds, which discards some information; the confidence intervals of RQ1 use all runs. The reliability differences of RQ1 are large compared with their intervals, except for precision and MCC on ClassEval, where AutoM2M and Single cannot be separated. Experiment 1 was pre-registered; its exploratory comparison of the revised and the original AutoM2M was not.

8 Conclusion

AutoM2M asks an LLM builder for a typed team of views and hybrid M2M transformations, run by AgentHOT, and admits the team only if a program can check that its parts fit. In 44,352 runs with seven open-weight models, AutoM2M teams had the fewest silent failures of all eight conditions and flagged more than 90% of their failures. Part of this came from abstaining, but admitted teams still failed silently about half as often as a single agent, and on function-level tasks their "done" was the most reliable signal of success. When they failed, their traces located the responsible binding of 96–98% of injected faults and named the fault class more than twice as often as transcript-based methods, and checked repairs matched rebuilding at lower cost. A check that takes milliseconds caught most independently seeded defects, which LLM critics could not tell from clean teams. These benefits come at a price: lower success, 7–16 times the output tokens of a single agent, many refused sessions for weak builders, and correct results that are not reported as done. AutoM2M is therefore not a way to solve more tasks, but a way to know more reliably which tasks were solved and why the others were not, following one principle: LLMs propose; programs decide.

Data Availability

The replication package contains the AutoM2M and AgentHOT implementation in both versions of Section **??** (lifter, builder prompts, checker, compiler, runtime, attribution and repair; `src/`); the typed-team fixtures and mutation operators (`teams/`, `evaluation/rq2/`); the Who&When audit and coding scripts (`evaluation/rq1/`); the experiment harness with the prompts of every condition (`evaluation/`); the Slurm pipeline, job scripts and environment records of every run (`hpc/`); the pre-registration of Experiment 1; the logs of fixes made during the study (`docs/experiment-log.md`, `hpc/logs/<run>/CHANGES.md`); all raw run records; and the analysis pipeline (`evaluation/analysis/analyze.py`) that produces every number, table and figure of this paper from `results/data/*.csv`. An archival copy of the package will receive a DOI at acceptance [DOI to be added].

Declarations

Funding. [To be completed.] **Conflict of interest.** The author declares no conflict of interest. **Ethical approval.** Not applicable: the study involves no human participants; all failure coding was done by LLMs. **Acknowledgements.** [To be completed.] **Use of generative AI.** LLMs are the object of study; their use in preparing the manuscript is [to be declared according to the journal's policy].

Appendix A Silent failures per model

Table A1 Silent-failure rate (% of runs that say done but fail) per model and condition, mean over three seeds; *adm.*: AutoM2M restricted to its admitted sessions. Models in the order of Figure **??**.

Model	Single	S.-Gate	Free	Critic	Schema	Typed-NC	AutoM2M	Typed-Ref	AutoM2M adm.
ClassEval									
Qwen2.5-Coder 32B	24.3	30.0	47.3	49.0	65.3	6.3	14.7	18.7	15.1
Devstral 24B	21.7	25.7	1.7	5.3	63.0	11.0	17.0	27.7	17.4
StarCoder2 15B	0.0	0.3	72.0	81.0	62.3	7.7	0.0	0.0	0.0
Gemma 3 27B	23.7	26.3	68.0	70.7	68.0	2.7	4.0	13.0	9.1
Qwen2.5-Coder 7B	19.3	22.3	58.7	65.0	31.0	2.3	5.0	19.7	8.0
Code Llama 13B	16.7	24.0	41.0	41.3	15.3	6.7	0.7	4.7	2.3
Granite-Code 20B	17.3	22.0	6.0	8.3	88.3	34.3	0.0	6.7	0.0
HumanEval+									
Qwen2.5-Coder 32B	10.8	11.2	18.9	19.7	16.1	5.5	6.3	9.8	6.6
Devstral 24B	20.3	18.9	6.9	8.3	24.0	8.1	11.6	19.1	11.9
StarCoder2 15B	0.0	1.2	90.0	90.2	43.5	15.0	0.0	0.0	0.0
Gemma 3 27B	17.1	17.5	27.8	28.7	22.0	7.1	6.9	18.3	11.1
Qwen2.5-Coder 7B	13.4	14.2	21.5	22.2	2.2	5.7	5.7	25.0	14.0
Code Llama 13B	46.1	50.4	25.8	24.6	16.9	11.4	7.7	22.4	16.1
Granite-Code 20B	35.0	36.8	6.5	5.5	66.5	36.0	0.8	15.2	3.8